

Документация ITS

Интеллектуальная система формирования торговых стратегий

Дата формирования: 2026-06-21

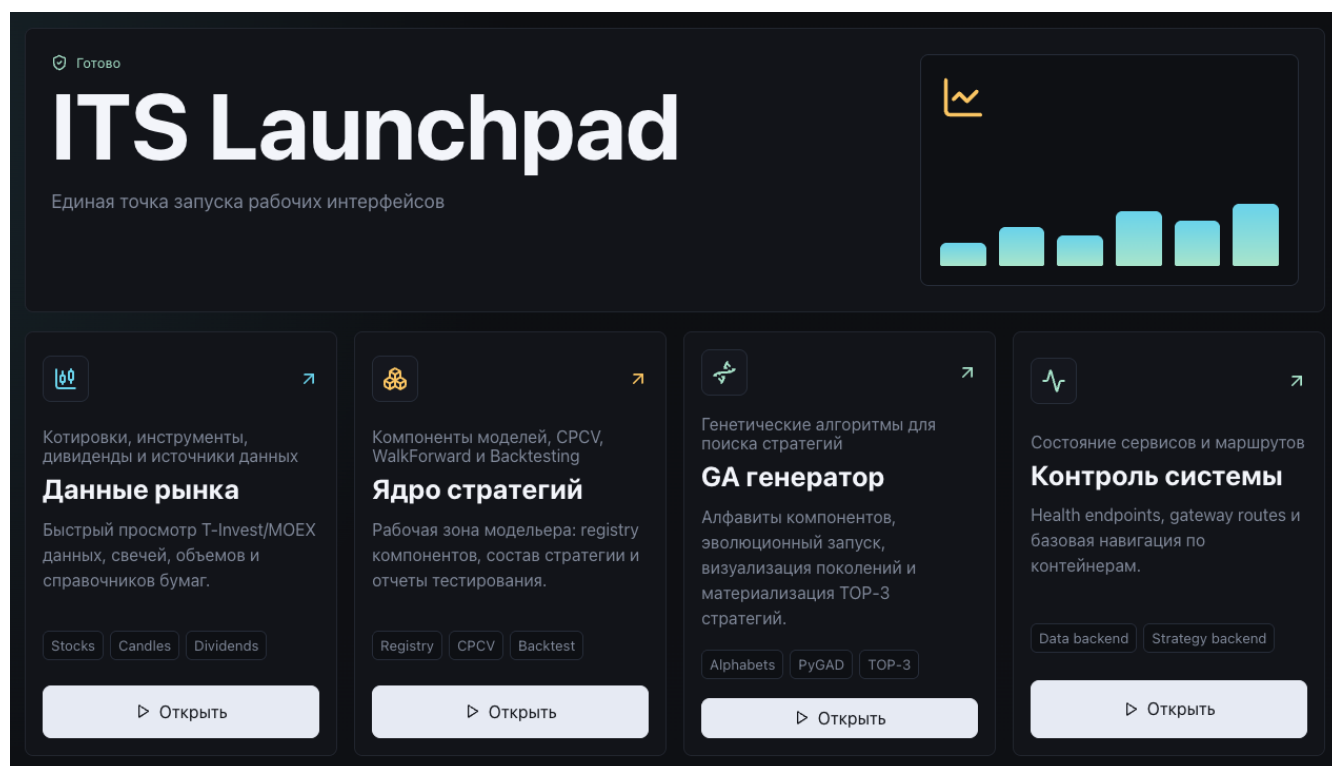
Содержание

1	Оглавление
2	Назначение и пользовательские роли
3	Установка, предварительные требования и запуск
4	Архитектура системы
5	Data Hub
6	Strategy Lab
7	GA Lab
8	Руководство модельера и разработчика компонентов
9	Методики тестирования и сравнения моделей
10	API и интеграции
11	Execution и технические сервисы
12	Эксплуатация, конфигурация и передача системы
13	Научная база и публикации автора
14	Глоссарий
15	Свидетельство о регистрации программы для ЭВМ

Документация ITS

ITS (Intelligent Trading Strategies) - интеллектуальная система формирования, тестирования и эволюционной генерации торговых стратегий для работы с биржевыми данными фондового, валютного и криптовалютного рынков.

Основной пользователь системы - финансовый модельер, квант-исследователь или аналитик, который проектирует торговые модели, проверяет их устойчивость на исторических данных и собирает новые стратегии из стандартизированных компонентов.



Состав документации

- 1 [Назначение и пользовательские роли](#)
- 2 [Установка, предварительные требования и запуск](#)
- 3 [Архитектура системы](#)
- 4 [Data Hub: данные, инструменты, дивиденды и custom bars](#)
- 5 [Strategy Lab: компоненты, модели и тестирование](#)
- 6 [GA Lab: генерация стратегий генетическим алгоритмом](#)
- 7 [Руководство модельера и разработчика компонентов](#)
- 8 [Методики тестирования и сравнения моделей](#)
- 9 [API и интеграции](#)
- 10 [Execution и технические сервисы](#)
- 11 [Эксплуатация, конфигурация и передача системы](#)
- 12 [Научная база и публикации автора](#)

13 Глоссарий

14 Свидетельство о регистрации программы для ЭВМ

Быстрый старт

1 Установить Docker и Docker Compose v2.

2 Создать файл `.env` в корне проекта и указать токен T-Invest:

```
tinvest_token=your_tinkoff_invest_token
```

1 Запустить систему одной командой:

```
docker compose up --build
```

1 Открыть главное окно:

<http://localhost:8080/launchpad/>

Основные интерфейсы

Интерфейс	URL	Назначение
Launchpad	/launchpad/	Главное окно запуска подсистем
Data Hub	/data/	Рыночные данные, инструменты, котировки, дивиденды
Strategy Lab	/strategies/	Реестр компонентов, сборка и тестирование моделей
GA Lab	/ga/	Генерация стратегий генетическим алгоритмом
Execution	/execution/	Брокерские счета, заявки, stop-заявки и запуск назначенных стратегий
Tech System	/tech/auth/	Вход, регистрация, роли, права и журнал действий
Data API	/api/data/docs	Swagger Data Backend
Strategy API	/api/strategies/docs	Swagger Strategy Backend
GA API	/api/ga/docs	Swagger GA Backend
Execution API	/api/execution/docs	Swagger Execution Backend
Tech API	/api/tech/docs	Swagger Tech System Backend
Event Log API	/api/event-log/docs	Swagger Event Log Backend

Что получает пользователь

Система дает финансовому модельеру единое рабочее пространство, в котором можно:

- подключать источники рыночных данных;
- исследовать инструменты, котировки, дивиденды и производные бары;
- создавать компоненты торговой модели: `pre-selection`, `signal`, `allocation`;

- собирать стандартизированные стратегии из этих компонентов;
- тестировать стратегии через CPCV, WalkForward и Backtesting;
- сравнивать стратегии по агрегированному рейтингу;
- задавать алфавиты генетического алгоритма и автоматически генерировать новые стратегии;
- материализовывать лучшие GA-кандидаты в Python-код в реестре моделей;
- просматривать брокерские счета T-Invest, создавать заявки и stop-заявки через Execution;
- управлять пользователями, ролями, JWT-сессиями и журналом действий через Tech System.

Статус текущей реализации

Текущая версия является исследовательско-лабораторной платформой для формирования, анализа, тестирования, генерации и контролируемого исполнения торговых стратегий. Система содержит инфраструктуру данных, UI, API, ядро стратегий, тестирование, GA-генератор, Execution-модуль, RBAC/JWT-аутентификацию, журнал событий и observability-профиль. Production-риск-лимиты и полностью автоматизированный order-management контур должны настраиваться и проверяться отдельно.

Назначение и пользовательские роли

Назначение системы

ITS предназначена для разработки торговых стратегий как набора стандартизированных, взаимозаменяемых компонентов. Система отделяет исследовательскую работу финансового модельера от инфраструктурных задач: загрузки данных, подготовки матриц доходностей, запуска тестов, визуализации результатов, сравнения моделей и генерации новых комбинаций.

Ключевая идея продукта - превратить создание торговых алгоритмов в управляемый инженерный процесс. Модельер описывает математическую и экономическую логику компонента, а платформа берет на себя:

- получение и нормализацию рыночных данных;
- регистрацию компонентов;
- сборку стратегий в единый pipeline;
- запуск устойчивых тестов;
- сохранение результатов;
- сравнение моделей;
- перебор и эволюционную генерацию стратегий;
- назначение выбранных стратегий на брокерские счета и контролируруемую отправку заявок.

Для кого система

Финансовый модельер

Основной пользователь. Его задачи:

- формировать гипотезы о поведении рынков;
- создавать селекторы активов, сигнальные модели и аллокаторы;
- проверять стратегии на исторических данных;
- сравнивать результаты;
- передавать лучшие стратегии в дальнейшую разработку или промышленный контур.

Квант-исследователь

Использует систему для исследования устойчивости факторов и портфельных правил:

- проверка разных режимов отбора активов;
- оценка стабильности на WalkForward-окнах;
- анализ переобучения через CPCV;
- сравнение метрик доходности и риска.

Разработчик интеграций

Расширяет систему новыми источниками данных:

- котировки;
- справочники инструментов;
- дивиденды;

- альтернативные бары;
- дополнительные фундаментальные или новостные признаки.

Технический заказчик или покупатель

Получает систему как готовую платформу, которую можно запустить одной командой, изучить через UI и расширять по документированным протоколам.

Администратор платформы

Управляет пользователями, ролями и правами доступа через Tech System, проверяет журнал действий и контролирует настройки Execution перед работой с брокерскими счетами.

Какие задачи решает система

1. Централизация данных

Data Hub подключает источники рыночной информации и предоставляет единый API для котировок, инструментов, валют, дивидендов и custom bars.

2. Унификация торговых моделей

Стратегия описывается как последовательность компонентов:

pre-selection -> signal -> allocation

Такой подход делает модели сравнимыми, расширяемыми и пригодными для автоматической генерации.

3. Быстрое тестирование гипотез

Strategy Lab позволяет выбрать модель, задать параметры теста и получить:

- метрики доходности и риска;
- графики equity curve;
- WalkForward OOS-кривую;
- состав портфеля по ребалансировкам;
- секторную агрегацию;
- события исполнения stop-loss и take-profit для полноценных торговых стратегий.

4. Генерация новых стратегий

GA Lab использует алфавиты компонентов как пространство поиска. Генетический алгоритм перебирает комбинации генов, оценивает кандидатов и сохраняет лучшие стратегии в [its/strategies/models](#).

5. Контур Execution и администрирования

Execution показывает настроенные счета T-Invest, портфель, позиции, заявки, stop-заявки и операции. Пользователь может создавать ручные заявки, назначать торговые стратегии на счет и запускать предварительный расчет заявок. Tech System отвечает за регистрацию, вход, JWT-сессии, роли, permissions и журнал пользовательских действий.

6. Использование LLM в разработке компонентов

Так как компоненты имеют четкие протоколы и небольшие интерфейсы, LLM можно использовать для быстрой генерации новых селекторов, сигналов и аллокаторов. Модельеру не требуется каждый раз описывать полный контур тестирования: достаточно реализовать новый компонент в установленном формате.

Принципы проектирования

Система построена вокруг следующих принципов:

- **стандартизированные модели** - стратегия имеет единый формат и воспроизводимый pipeline;
- **Open-Closed Principle** - новые компоненты добавляются без изменения существующего ядра;
- **Interface Segregation Principle** - селекторы, сигналы, аллокаторы, trading policies и источники данных имеют узкие интерфейсы;
- **проверяемость** - ключевые функции покрываются тестами и доступны через API;
- **расширяемость** - новые источники данных, компоненты и гены GA добавляются в выделенные директории;
- **воспроизводимость** - результаты тестов кэшируются, а GA-кандидаты материализуются в Python-файлы.

Пользовательский результат

После работы в системе пользователь получает:

- список доступных активов и рыночных данных;
- сформированные торговые модели;
- отчеты CPCV, WalkForward и Backtesting;
- рейтинг стратегий;
- файлы сгенерированных стратегий;
- назначение стратегий на брокерские счета и заявки Execution;
- журнал действий и роли пользователей;
- основу для дальнейшей научной, продуктовой или инвестиционно-аналитической работы.

Установка, предварительные требования и запуск

Минимальные требования

Для обычного запуска системы пользователю достаточно:

- Docker Engine или Docker Desktop;
- Docker Compose v2;
- Git для получения исходного кода;
- доступ к интернету для сборки образов и обращения к источнику данных;
- токен T-Invest API для загрузки рыночных данных;
- идентификаторы брокерских счетов T-Invest для Execution, если нужен контур исполнения;
- свободный порт 8080 или другой порт, заданный через `ITS_GATEWAY_PORT`.

Рекомендуемые ресурсы рабочей станции:

- 4 CPU и выше;
- 8 GB RAM и выше;
- 5 GB свободного места для образов, кэшей данных и результатов тестов.

Технологические требования для разработки

Если система запускается не только через Docker, а также дорабатывается локально, потребуются:

- Python 3.12.x;
- Poetry;
- Node.js 20+;
- npm;
- современный браузер.

Основные backend-библиотеки:

- FastAPI и Uvicorn для API-сервисов;
- pandas и NumPy для подготовки данных;
- scikit-learn pipeline-интерфейс для компонентов;
- skfolio для портфельной оптимизации и model selection;
- vectorbt для backtesting;
- PyGAD для генетического алгоритма;
- httpx, aiohttp, aiometer, async-lru для HTTP и асинхронной загрузки;
- t-tech-investments для интеграции с T-Invest.

Основные frontend-библиотеки:

- Vue 3;
- Vite;
- TypeScript;

- lucide-vue-next;
- ECharts в Data UI.

Настройка токена

В корне проекта создать или обновить файл `.env`:

```
tinvest_token=your_tinkoff_invest_token
```

Также поддерживаются переменные:

```
TINVEST_TOKEN=your_tinkoff_invest_token
TINKOFF_INVEST_API_TOKEN=your_tinkoff_invest_token
```

Токен нужен Data Backend для получения инструментов, котировок, валют и дивидендов через T-Invest API. Execution использует `EXECUTION_TINVEST_TOKEN`, если он задан, иначе те же переменные токена.

Для Execution можно задать список счетов:

```
EXECUTION_TINVEST_ACCOUNT_IDS=account_id_1,account_id_2
EXECUTION_TINVEST_ACCOUNTS=account_id_1:Main,account_id_2:IIS
EXECUTION_ORDER_SUBMISSION_MODE=stub
```

`EXECUTION_ORDER_SUBMISSION_MODE=stub` проверяет заявки локально и не отправляет их брокеру. Значение по умолчанию - `real`.

Запуск всей системы

Из корня проекта:

```
docker compose up --build
```

Команда собирает и запускает:

- `data-backend`;
- `data-ui`;
- `strategy-backend`;
- `strategy-ui`;
- `ga-backend`;
- `ga-ui`;
- `execution-backend`;
- `execution-ui`;
- `tech-system-backend`;
- `tech-system-ui`;
- `event-log-backend`;
- `launchpad-ui`;
- PostgreSQL для прикладных данных и отдельный PostgreSQL для журнала событий;
- `nginx-gateway`.

URL после запуска

Компонент	URL
Главное окно	http://localhost:8080/launchpad/

Компонент	URL
Data Hub	http://localhost:8080/data/
Strategy Lab	http://localhost:8080/strategies/
GA Lab	http://localhost:8080/ga/
Execution	http://localhost:8080/execution/
Tech System / Auth	http://localhost:8080/tech/auth/
Документация	http://localhost:8080/docs/
Data API Swagger	http://localhost:8080/api/data/docs
Strategy API Swagger	http://localhost:8080/api/strategies/docs
GA API Swagger	http://localhost:8080/api/ga/docs
Execution API Swagger	http://localhost:8080/api/execution/docs
Tech API Swagger	http://localhost:8080/api/tech/docs
Event Log API Swagger	http://localhost:8080/api/event-log/docs

Если порт **8080** занят:

```
ITS_GATEWAY_PORT=8090 docker compose up --build
```

Тогда главное окно будет доступно по адресу:

```
http://localhost:8090/launchpad/
```

Проверка работоспособности

После старта открыть:

```
http://localhost:8080/health
```

Ожидаемый ответ:

```
ok
```

Также можно проверить API:

```

http://localhost:8080/api/data/health
http://localhost:8080/api/strategies/health
http://localhost:8080/api/ga/health
http://localhost:8080/api/execution/health
http://localhost:8080/api/tech/health
http://localhost:8080/api/event-log/health

```

Ожидаемый JSON:

```
{"status":"ok"}
```

Данные и кэши

Docker Compose создает именованные volumes:

Volume	Назначение
<code>t-invest-cache</code>	кэш котировок, справочников и дивидендов
<code>strategy-test-cache</code>	сохраненные CPCV, WalkForward и Backtesting отчеты

Volume	Назначение
ga-cache	сохраненные GA-запуски
postgres-data	пользователи, роли, назначения стратегий и прикладное состояние
event-log-postgres-data	журнал событий пользовательских и API-действий

GA также записывает материализованные стратегии в локальную папку:

`its/strategies/models`

Типовые проблемы запуска

Не задан T-Invest токен

Data Backend вернет ошибку 503 с сообщением о необходимости задать `tinvest_token`, `TINVEST_TOKEN` или `TINKOFF_INVEST_API_TOKEN`.

Решение: проверить `.env` в корне проекта и перезапустить контейнеры.

Порт 8080 занят

Решение:

`ITS_GATEWAY_PORT=8090 docker compose up --build`

Нет котировок для выбранного периода

Причины:

- инструмент не найден;
- выбран слишком ранний период;
- источник данных не вернул свечи;
- неверный `class_code` или тип инструмента.

Решение: проверить тикер, FIGI, `class_code`, интервал и даты в Data Hub.

Слишком долгий GA-запуск

Причины:

- большая популяция;
- много поколений;
- длинный период данных;
- большое количество активов;
- сложные CPCV и WalkForward параметры.

Решение: уменьшить `num_generations`, `sol_per_pop`, число активов или длину периода.

Execution не показывает счета

Причины:

- не задан токен T-Invest;
- не задан `EXECUTION_TINVEST_ACCOUNT_IDS` или `EXECUTION_TINVEST_ACCOUNTS`;
- указанный счет не возвращается T-Invest API для текущего токена.

Решение: проверить `.env`, права токена и перезапустить контейнеры.

Архитектура системы

Общий вид

ITS состоит из шести пользовательских интерфейсов, шести backend-сервисов, Python-ядра моделей, подсистемы загрузки данных, GA-движка, Execution-контура, Tech System, журнала событий и observability-профиля.

flowchart LR

User["Финансовый модельер"] --> Gateway["nginx-gateway
единая точка входа"]

Gateway --> Launchpad["launchpad-ui
/launchpad/"]

Gateway --> DataUI["data-ui
/data/"]

Gateway --> StrategyUI["strategy-ui
/strategies/"]

Gateway --> GAUI["ga-ui
/ga/"]

Gateway --> ExecutionUI["execution-ui
/execution/"]

Gateway --> TechUI["tech-system-ui
/tech/auth/"]

Gateway --> DataAPI["data-backend
/api/data/"]

Gateway --> StrategyAPI["strategy-backend
/api/strategies/"]

Gateway --> GAAP["ga-backend
/api/ga/"]

Gateway --> ExecutionAPI["execution-backend
/api/execution/"]

Gateway --> TechAPI["tech-system-backend
/api/tech/"]

Gateway --> EventLogAPI["event-log-backend
/api/event-log/"]

StrategyAPI --> DataAPI

GAAP --> DataAPI

ExecutionAPI --> DataAPI

ExecutionAPI --> Broker["T-Invest broker API"]

StrategyAPI --> Core["its/strategies
ядро компонентов и моделей"]

GAAP --> GA["its/ga
алфавиты и PyGAD"]

GA --> Models["its/strategies/models
материализованные стратегии"]

TechAPI --> Auth["its/tech_system/auth
RBAC и JWT"]

DataAPI --> EventDB

StrategyAPI --> EventDB

GAAP --> EventDB

ExecutionAPI --> EventDB

TechAPI --> EventDB

DataAPI --> Loader["its/data_loader
источники данных"]

Loader --> TInvest["T-Invest API"]

TechAPI --> AppDB["postgres
пользователи и роли"]

EventLogAPI --> EventDB["event-log-postgres
аудит действий"]

Контейнеры Docker Compose

Сервис	Путь	Назначение
nginx-gateway	infra/nginx	маршрутизация UI, API и документации
launchpad-ui	ui/launchpad-ui	стартовый экран системы
data-ui	ui/data-ui	интерфейс данных
strategy-ui	ui/strategy-ui	интерфейс моделей и тестов
ga-ui	its/ui/ga-ui	интерфейс GA-генерации
execution-ui	ui/execution-ui	интерфейс брокерских счетов и заявок
tech-system-ui	ui/tech-system-ui	интерфейс входа, регистрации и ролей
data-backend	services/data_backend	API источников данных

Сервис	Путь	Назначение
strategy-backend	services/strategy_backend	API реестра моделей и тестов
ga-backend	its/services/ga_backend	API генетического алгоритма
execution-backend	its/services/execution_backend	API брокерских счетов, заявок и назначенных стратегий
tech-system-backend	services/tech_system_backend	API аутентификации, пользователей, ролей и permissions
event-log-backend	services/event_log_backend	API чтения журнала пользовательских и API-действий
postgres	Docker volume postgres-data	прикладная БД: пользователи, роли, назначения стратегий
event-log-postgres	Docker volume event-log-postgres-data	отдельная БД журнала событий

Маршрутизация gateway

nginx-gateway открывает наружу один порт и маршрутизирует запросы:

Внешний путь	Внутренний сервис
/	redirect на /launchpad/
/launchpad/	launchpad-ui
/data/	data-ui
/strategies/	strategy-ui
/ga/	ga-ui
/execution/	execution-ui
/tech/	tech-system-ui
/docs/	Markdown-документация из папки docs
/api/data/	data-backend
/api/strategies/	strategy-backend
/api/ga/	ga-backend
/api/execution/	execution-backend
/api/tech/	tech-system-backend
/api/event-log/	event-log-backend
/grafana/	grafana, только профиль observability
/opensearch-api/	opensearch, только профиль observability

Backend-архитектура

Data Backend

Путь:

[services/data_backend](#)

Основные функции:

- health-check;
- список источников данных;
- справочник акций;
- справочник валют;
- загрузка свечей;
- построение custom gold bars;
- загрузка дивидендов;
- нормализация и кэширование ответов.

Data Backend использует код загрузчиков из:

[its/data_loader](#)

Strategy Backend

Путь:

[services/strategy_backend](#)

Основные функции:

- реестр компонентов;
- реестр моделей ядра торговой стратегии;
- реестр полноценных торговых стратегий;
- детальная структура выбранной модели;
- запуск и чтение CPCV-тестов;
- запуск и чтение WalkForward-тестов;
- запуск и чтение Backtesting-тестов;
- сравнение стратегий по последним сохраненным тестам.

GA Backend

Путь:

[its/services/ga_backend](#)

Основные функции:

- чтение алфавитов генетического алгоритма;
- запуск GA-задачи в фоне;
- мониторинг статуса запуска;
- сохранение истории запусков;
- материализация TOP-N стратегий в Python-код.

Execution Backend

Путь:

`its/services/execution_backend`

Основные функции:

- health-check с режимом отправки заявок;
- чтение настроенных брокерских счетов T-Invest;
- обзор счета: портфель, позиции, заявки, stop-заявки, операции и маржинальные атрибуты;
- отправка обычных и stop-заявок в `real` или `stub` режиме;
- получение последней цены и WebSocket order book;
- назначение trading strategies на счет и запуск preview/исполнения назначенной стратегии.

Tech System Backend

Путь:

`services/tech_system_backend`

Основные функции:

- регистрация, вход, refresh и logout;
- JWT access/refresh tokens;
- RBAC: роли, permissions и назначение ролей пользователям;
- заявки на роли и их согласование;
- аудит auth- и role-событий.

Event Log Backend

Путь:

`services/event_log_backend`

Основные функции:

- чтение append-only журнала действий;
- фильтрация по сервису, пользователю, HTTP-методу, пути, IP и датам;
- выдача доступных значений фильтров;
- отдельное хранение в `event-log-postgres`.

Observability

Профиль `observability` добавляет Prometheus, Alertmanager, Grafana, OpenSearch, OpenSearch Dashboards, Fluent Bit, OpenTelemetry Collector, exporters, cAdvisor и GlitchTip. Backend-сервисы устанавливают общий middleware `its/observability` и отдают метрики на `/metrics`.

Frontend-архитектура

Все UI написаны на Vue 3, TypeScript и Vite.

UI	Путь	Роль
Launchpad	<code>ui/launchpad-ui</code>	плитки запуска подсистем
Data UI	<code>ui/data-ui</code>	работа с рыночными данными

UI	Путь	Роль
Strategy UI	ui/strategy-ui	работа с моделями, тестами и сравнением
GA UI	its/ui/ga-ui	настройка и мониторинг генетического алгоритма
Execution UI	ui/execution-ui	счета, портфель, заявки, order book и назначенные стратегии
Tech System UI	ui/tech-system-ui	вход, регистрация, роли, permissions и журнал событий

Интерфейсы обращаются к API через gateway, поэтому пользователю не нужно знать внутренние адреса контейнеров.

Кодовое ядро стратегий

Ключевые директории:

Путь	Назначение
its/strategies/core/selectors	предварительная фильтрация активов
its/strategies/core/signals	сигнальные модели
its/strategies/core/optimization	аллокаторы портфеля
its/strategies/core/types	базовые типы и протоколы
its/strategies/models	готовые модели ядра торговой стратегии
its/strategies_model/core	полноценная торговая стратегия и политики выхода
its/strategies_model/model	собранные trading strategies
its/strategies/testing	CPCV, WalkForward, Backtesting, comparison
its/ga/alphabets	алфавиты генов для GA
its/ga	registry, engine, materialization
its/execution	доменная логика Execution, схемы заявок, интеграция T-Invest
its/tech_system/auth	RBAC, JWT, схемы и маршруты технической подсистемы
its/event_log	middleware, repository, schemas и router журнала событий
its/observability	logging, metrics, tracing и error tracking
its/db	модели SQLAlchemy и Alembic-миграции
its/authz	единый контекст авторизации для backend-сервисов

Pipeline стратегии

Ядро торговой стратегии использует pipeline:

Каждый шаг можно заменить новым компонентом при соблюдении интерфейса. Такой подход позволяет:

- добавлять компоненты без изменения существующих моделей;
- комбинировать компоненты вручную;
- комбинировать компоненты автоматически через GA;
- использовать единый контур тестирования для всех стратегий.

Входные и выходные данные

Входные данные

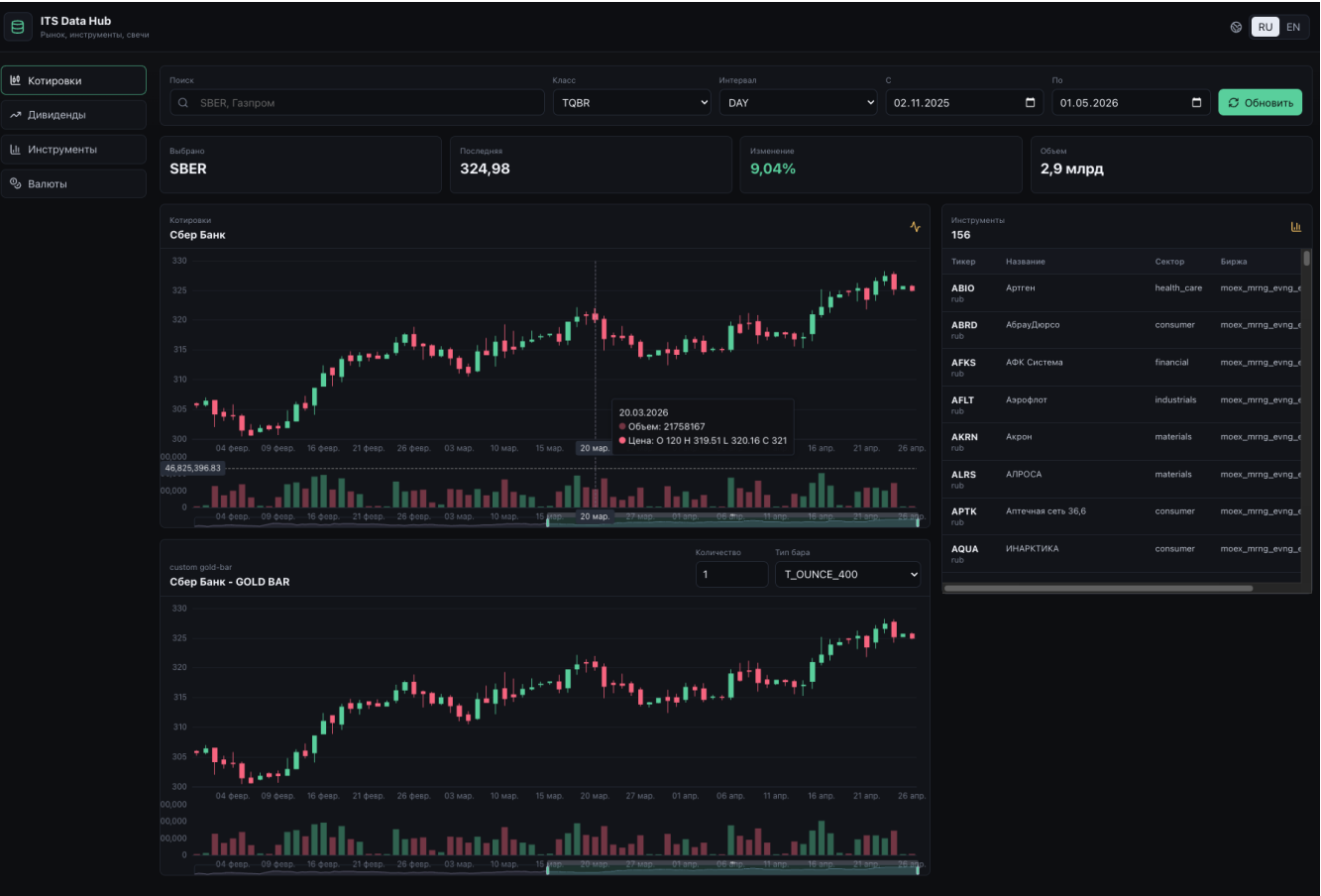
- справочник инструментов;
- исторические свечи OHLCV;
- дивиденды;
- параметры тестирования;
- классы моделей;
- алфавиты GA;
- JWT-сессии и права пользователей;
- идентификаторы брокерских счетов и параметры заявок;
- события HTTP/API для журнала.

Выходные данные

- очищенные таблицы данных;
- графики и таблицы в UI;
- JSON-отчеты CPCV, WalkForward и Backtesting;
- агрегированный рейтинг моделей;
- материализованные Python-классы сгенерированных стратегий;
- состояние счетов, портфеля, заявок и назначений стратегий;
- записи журнала событий;
- метрики, структурные логи и ошибки observability-контура;
- кэши данных и тестов.

Data Hub

Data Hub - подсистема данных ITS. Она отвечает за подключение источников, получение рыночной информации, нормализацию данных и предоставление единого API для остальных сервисов.



Назначение

Data Hub решает три группы задач:

- предоставляет модельеру визуальный интерфейс для просмотра данных;
- предоставляет backend API для Strategy Lab, GA Lab и Execution;
- изолирует код интеграций с внешними источниками от логики стратегий.

Основные файлы

Путь	Назначение
services/data_backend/app/main.py	FastAPI-приложение Data Backend
ui/data-ui	Vue-интерфейс Data Hub
its/data_loader	слой загрузчиков и адаптеров источников данных
its/data_loader/t_invest_data_readers	подключенный источник T-Invest
its/data_loader/custom_bar/gold_bar	построение custom gold bars

Доступные данные в базовой версии

Базовая версия подключает T-Invest как источник данных и предоставляет:

- российские акции;
- валютные инструменты;
- свечи OHLCV;
- информацию о тикерах;
- сектора, биржи, валюты, классы инструментов;
- дивиденды;
- RSS-новости из настроенных источников;
- Monte Carlo-сценарии цены закрытия;
- custom gold bars, пересчитанные через золото.

Разделы UI

Котировки

Раздел предназначен для просмотра свечей выбранного инструмента.

Пользователь выбирает:

- класс инструмента, например TQBR;
- интервал свечей;
- дату начала;
- дату окончания;
- инструмент.

UI отображает:

- свечной график;
- последнюю цену;
- процентное изменение за период;
- число свечей;
- объем.

Дивиденды

Раздел показывает дивидендную историю выбранной бумаги:

- дата объявления;
- последний день покупки;
- дата фиксации;
- дата выплаты;
- размер дивиденда;
- доходность;
- тип и регулярность выплаты.

Дивиденды используются не только для анализа, но и как входные данные для компонентов стратегии, например `DividendHistorySelector`.

Инструменты

Раздел справочника акций. Пользователь может искать бумаги по названию или тикеру и видеть:

- FIGI;
- тикер;
- ISIN;
- название;
- сектор;
- биржу;
- валюту;
- страну риска;
- лот;
- торговый статус;
- доступность покупки и продажи.

Валюты

Раздел валютных инструментов. Используется в том числе для `GLDRUB_TOM`, который нужен при построении custom gold bars.

Custom gold-bar

Data Hub умеет строить альтернативные бары через стоимость золота. Пользователь задает:

- базовый инструмент;
- золотой инструмент, по умолчанию `GLDRUB_TOM`;
- количество золотых единиц;
- тип бара, например `GRAM`, `T_OUNCE`, `KG`, `T_OUNCE_400`;
- период и интервал.

Результат - OHLCV-бары, пересчитанные через стоимость выбранного количества золота.

Data Backend API

Все запросы снаружи идут через gateway:

`/api/data/`

Основные endpoints:

Endpoint	Назначение
<code>GET /api/data/health</code>	проверка статуса
<code>GET /api/data/sources</code>	список источников
<code>GET /api/data/stocks</code>	список акций
<code>GET /api/data/currencies</code>	список валют

Endpoint	Назначение
GET /api/data/instruments	универсальный справочник торгуемых инструментов
GET /api/data/prices	свечи OHLCV
GET /api/data/custom-gold-bars	custom gold bars
GET /api/data/monte-carlo	сценарии цены закрытия по Monte Carlo
GET /api/data/dividends	дивиденды
GET /api/data/rss	новости из RSS-хранилища
POST /api/data/rss/load	загрузка RSS-источников в БД
GET /api/data/docs	Swagger

Поддерживаемые интервалы свечей

Data Backend принимает интервалы:

- CANDLE_INTERVAL_1_MIN;
- CANDLE_INTERVAL_5_MIN;
- CANDLE_INTERVAL_15_MIN;
- CANDLE_INTERVAL_HOUR;
- CANDLE_INTERVAL_DAY;
- CANDLE_INTERVAL_WEEK;
- CANDLE_INTERVAL_MONTH.

Кэширование

Загрузчики T-Invest используют файловый кэш внутри `its/data` в контейнере. В Docker Compose эта директория вынесена в volume:

```
t-invest-cache
```

Кэш снижает число обращений к внешнему API, ускоряет повторные тесты и позволяет переиспользовать уже загруженные периоды.

Добавление нового источника данных

Новые интеграции добавляются в:

```
its/data_loader
```

Рекомендуемый порядок:

- 1 Создать адаптер источника.
- 2 Нормализовать выходные поля к формату, который ожидает Data Backend.
- 3 Добавить use case, если требуется отдельная бизнес-логика.
- 4 Расширить Data Backend endpoint или добавить новый.
- 5 Обновить Data UI, если данные должны быть доступны пользователю визуально.

Требования к данным для стратегий

Стратегии и тесты ожидают, что свечи содержат как минимум:

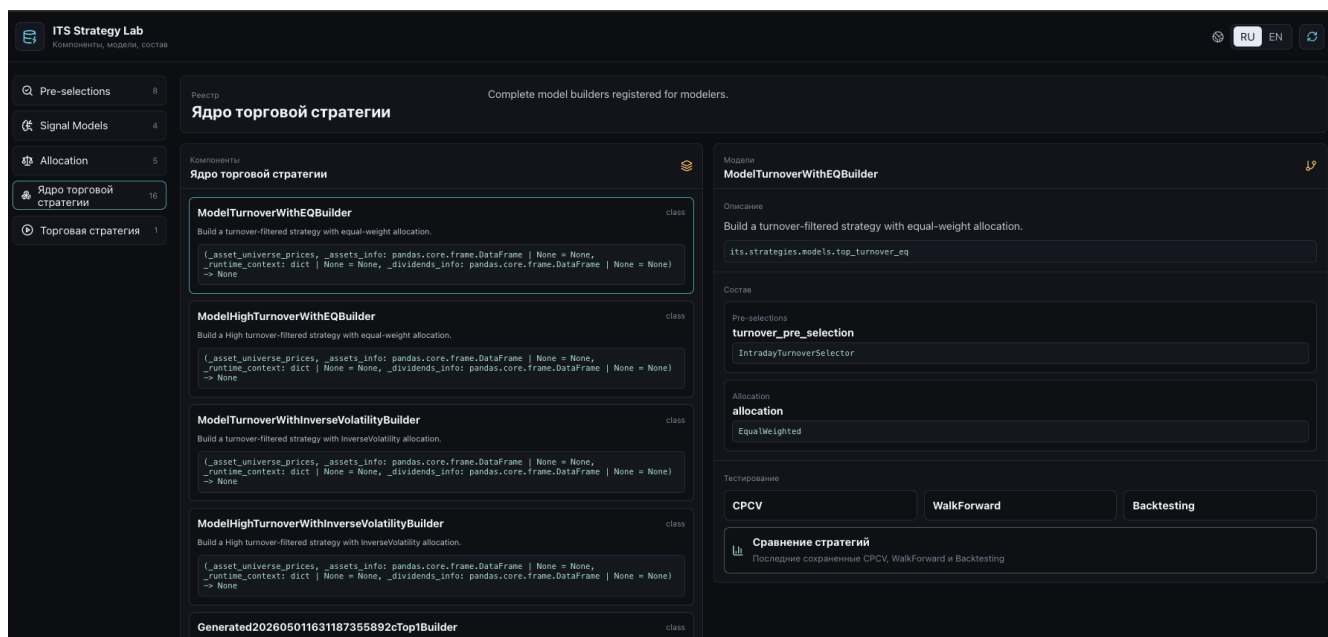
Поле	Назначение
time	дата и время свечи
ticker	тикер инструмента
figi	FIGI
open	цена открытия
high	максимум
low	минимум
close	цена закрытия
volume	объем
is_complete	признак завершенной свечи

Для дивидендных селекторов нужны:

Поле	Назначение
ticker	тикер
last_buy_date	последний день покупки для получения дивиденда

Strategy Lab

Strategy Lab - основная рабочая зона финансового модельера. Здесь отображаются компоненты, готовые модели, состав стратегии, тесты CPCV, WalkForward, Backtesting и сравнение моделей.



Назначение

Strategy Lab нужен для:

- просмотра зарегистрированных компонентов;
- выбора готовой модели;
- понимания состава модели;
- запуска тестов;
- чтения сохраненных результатов;
- сравнения стратегий;
- анализа состава портфеля и секторной экспозиции;
- отметки trading strategies как готовых к Execution.

Основные файлы

Путь	Назначение
services/strategy_backend	FastAPI backend Strategy Lab
ui/strategy-ui	Vue UI Strategy Lab
its/strategies/core	компоненты ядра
its/strategies/models	готовые модели ядра
its/strategies/testing	тестовые методики

Путь	Назначение
its/strategies_model	полноценные торговые стратегии

Структура стратегии

Ядро стратегии состоит из трех последовательных шагов:

pre-selection -> signal -> allocation

Pre-selection

Первичная фильтрация активов. Селектор получает матрицу цен или доходностей и формирует маску активов, которые допускаются к дальнейшему анализу.

Примеры в текущей версии:

- `IntradayTurnoverSelector` - отбор по обороту;
- `CrossSectionalMomentumSelector` - отбор по cross-sectional momentum;
- `DividendHistorySelector` - отбор бумаг с дивидендной историей;
- `SectorSelector` - отбор по сектору;
- `TrendSelector` и `TrendThresholdSelector` - отбор по тренду;
- `KeepAllSelector` - пропуск без фильтрации;
- `SafeEmptySelector` - обертка, защищающая pipeline от пустой выборки.

Signal

Сигнальный слой применяет дополнительную логику выбора или ранжирования после pre-selection.

Примеры:

- `KeepAllSignal` - пропускает все активы;
- `PriceBreakoutSignal` - выбирает активы с пробоем максимума;
- `SMACrossSignal` - сигнал пересечения скользящих средних;
- `TwoCandlePositiveTrendSignal` - сигнал положительной динамики последних свечей.

Allocation

Аллокатор распределяет веса между выбранными активами.

Примеры:

- `EqualWeighted`;
- `InverseVolatility`;
- `HierarchicalRiskParity`;
- `CVaR`;
- `CVaRHighRisk`.

Аллокаторы используют skfolio и собственные расширения.

Вкладки Strategy Lab

Pre-selections

Показывает зарегистрированные селекторы. Для каждого элемента отображаются:

- имя класса;
- модуль;
- описание;
- сигнатура конструктора;
- параметры;
- путь к исходному коду.

Signal Models

Показывает зарегистрированные сигнальные модели с теми же атрибутами.

Allocation

Показывает зарегистрированные аллокаторы.

Ядро торговой стратегии

Ключевая вкладка. Здесь отображаются модели из:

[its/strategies/models](#)

Для выбранной модели пользователь видит:

- название;
- описание;
- состав pipeline;
- доступные тесты;
- список сохраненных отчетов;
- кнопки запуска CPCV, WalkForward, Backtesting.

Торговая стратегия

Полная торговая стратегия включает:

- ядро портфельной модели;
- правила входа и выхода;
- stop-loss;
- take-profit;
- дополнительные политики жизненного цикла позиции.

Пример текущей стратегии:

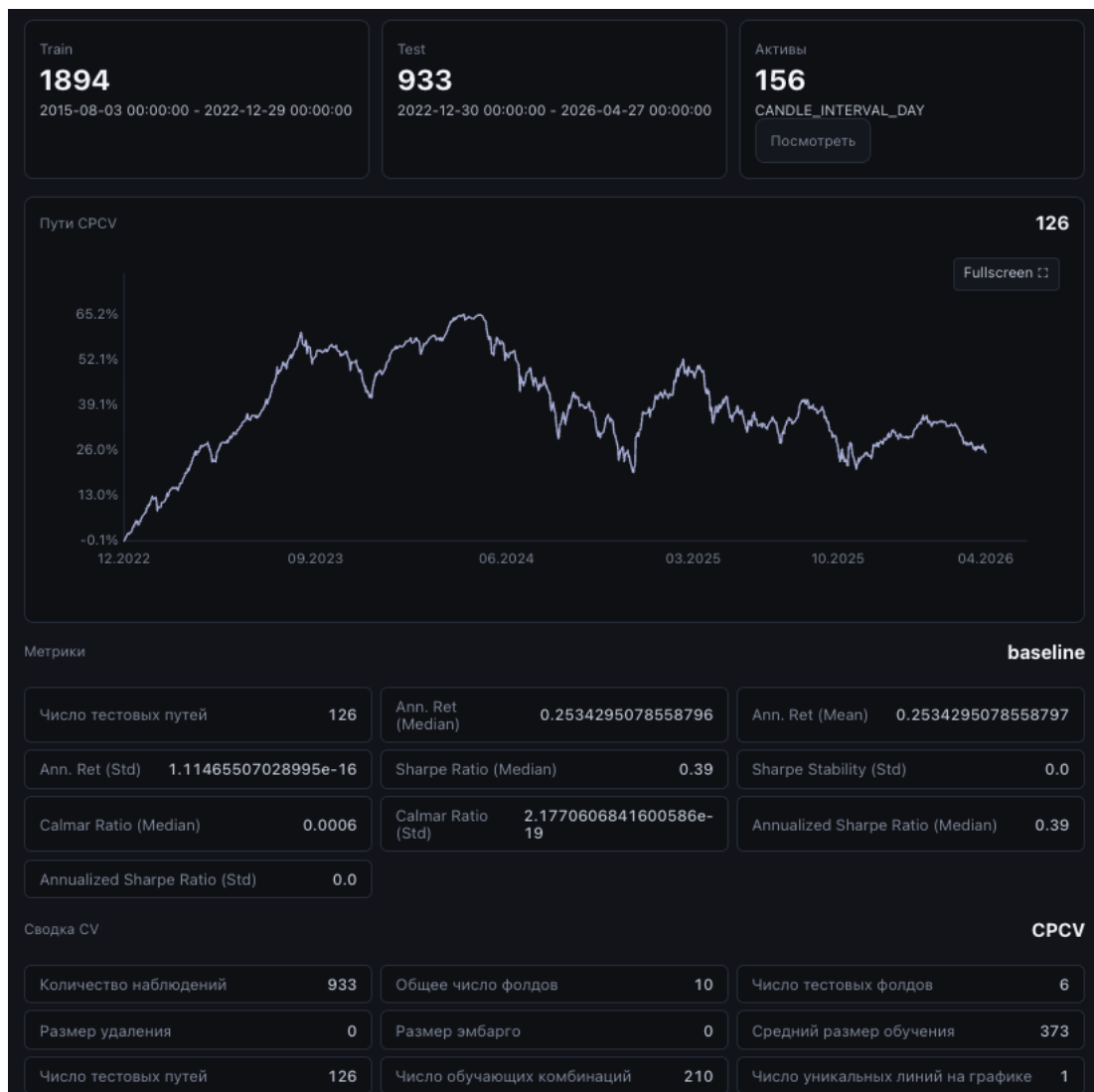
[TurnoverEqStopLoss1TakeProfit3Builder](#)

Она использует ядро [ModelTurnoverWithEQBuilder](#), стоп-лосс 1% и тейк-профит 3%.

Для передачи стратегии в Execution используется production state: стратегия помечается как [prod-ready](#), после чего она доступна для назначения на брокерский счет в Execution UI.

CPCV

CPCV запускается в отдельном модальном окне.



Пользователь задает:

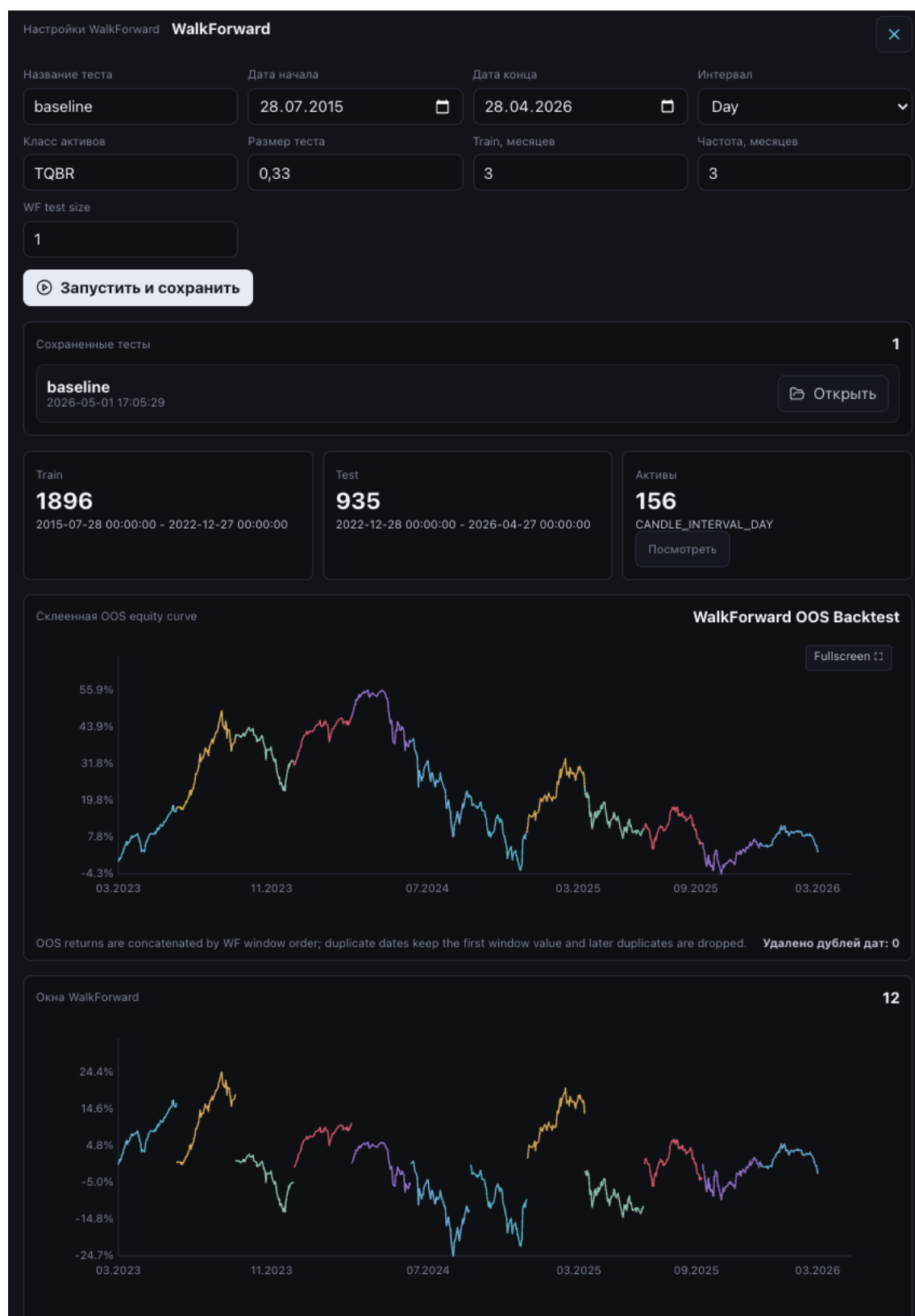
- название теста;
- дату начала;
- дату окончания;
- интервал;
- класс активов;
- `n_folds`;
- `n_test_folds`.

Результат:

- summary CPCV;
- метрики по test paths;
- графики кумулятивной доходности путей;
- список активов;
- сохраненный JSON-отчет.

WalkForward

WalkForward запускается в отдельном модальном окне.



Пользователь задает:

- название теста;
- период данных;

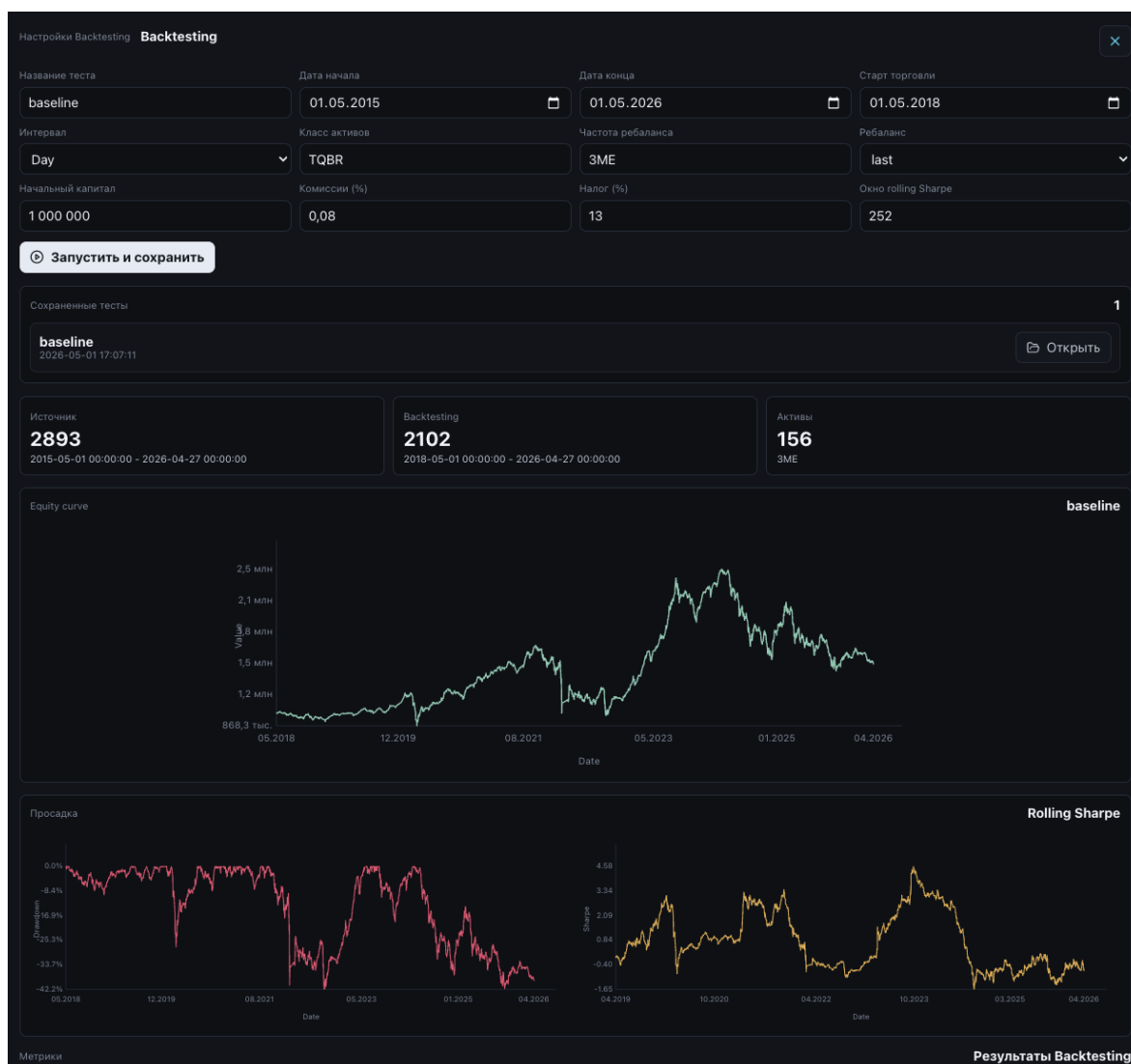
- долю OOS-теста;
- размер train-окна в месяцах;
- частоту сдвига окна;
- размер WF test;
- класс активов и интервал.

Результат:

- список WalkForward-окон;
- метрики;
- графики отдельных окон;
- склеенная OOS equity curve;
- число удаленных дублей дат при склейке;
- сохраненный JSON-отчет.

Backtesting

Backtesting запускается в отдельном модальном окне.



Пользователь задает:

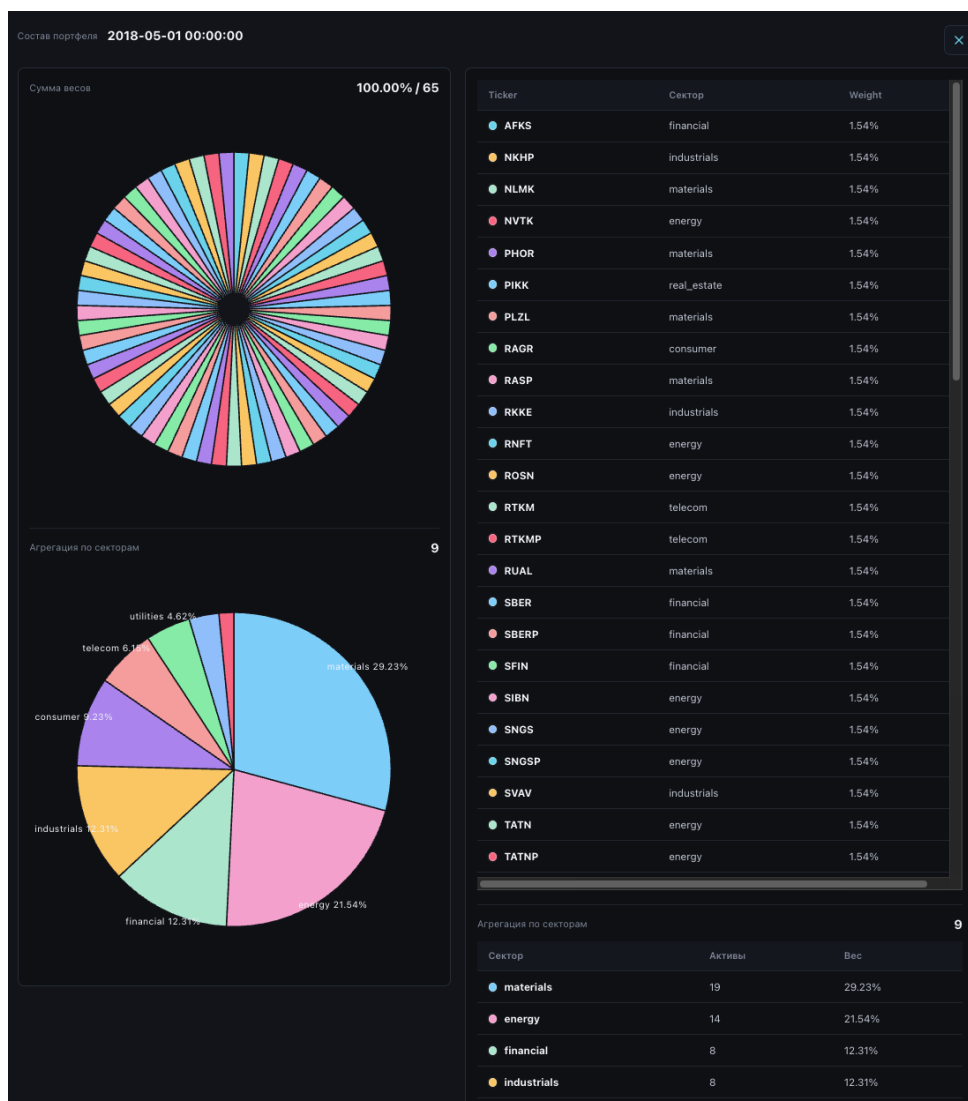
- период данных;
- дату начала торговли;
- частоту ребаланса;
- правило даты ребаланса;
- начальный капитал;
- комиссии;
- проскальзывание;
- налоговую ставку;
- окно rolling Sharpe.

Результат:

- equity curve;
- drawdown curve;
- rolling Sharpe;
- таблица метрик vectorbt;
- summary доходности и риска;
- веса портфеля на ребалансировках;
- события stop-loss и take-profit для полноценных торговых стратегий.

Состав портфеля

В результатах Backtesting можно открыть модальное окно состава портфеля.



Доступно:

- какие бумаги вошли в портфель на каждой ребалансировке;
- вес каждой бумаги;
- сумма весов;
- секторная агрегация;
- круговая диаграмма секторной экспозиции.

Сравнение моделей

Сравнение стратегий открывается отдельным модальным окном.

Последние сохраненные CPCV, WalkForward и Backtesting

Сравнение стратегий

Обновить сравнение

2026-05-01 17:33:33

Моделей в рейтинге

2

Пропущено

14

Рекомендация

ModelTurnoverWithEQBuilder

TOTAL_SCORE: 15.00

Сравнение стратегий

2

#	Моделей	TOTAL_SCORE	WF Return	WF Calmar	Robustness Delta	Sharpe Stability	Побед по метрикам	Backtest Return	Backtest Sharpe
1	ModelTurnoverWithEQBuilder	15.00	2.89%	0,0002	22.45%	0.00%	9.00	49.16%	0,4084
2	ModelHighTurnoverWithEQBuilder	7.00	1.05%	0,0022	5.39%	0.00%	1.00	-38.56%	-1,0238

Последние тесты

ModelTurnoverWithEQBuilder

CPCV / baseline

2026-05-01 17:04:06

ModelTurnoverWithEQBuilder_baseline_cpcv.json

WalkForward / baseline

2026-05-01 17:05:29

ModelTurnoverWithEQBuilder_baseline_walk_forward.json

Backtesting / baseline

2026-05-01 17:07:11

ModelTurnoverWithEQBuilder_baseline_backtest.json

Победители Backtesting

10

Total Return	ModelTurnoverWithEQBuilder - 0,4916
Коэффициент Шарпа	ModelTurnoverWithEQBuilder - 0,4084
Коэффициент Сортино	ModelTurnoverWithEQBuilder - 0,5216
Коэффициент Калмара	ModelTurnoverWithEQBuilder - 0,1705
Omega Ratio	ModelTurnoverWithEQBuilder - 1,0735
Win Rate [%]	ModelTurnoverWithEQBuilder - 72,0368
Profit Factor	ModelTurnoverWithEQBuilder - 8,2689
Expectancy	ModelTurnoverWithEQBuilder - 924,4099
Max Drawdown	ModelTurnoverWithEQBuilder - -0,4217
Total Fees Paid	ModelHighTurnoverWithEQBuilder - 1 216,7784

Пропущено

14

ModelTurnoverWithInverseVolatilityBuilder

cpcv, walk_forward

ModelHighTurnoverWithInverseVolatilityBuilder

cpcv, walk_forward

Generated202605011631187355892cTop1Builder

cpcv, walk_forward, backtesting

Generated202605011631187355892cTop2Builder

cpcv, walk_forward, backtesting

Generated202605011631187355892cTop3Builder

cpcv, walk_forward, backtesting

Generated20260501164633E4fd7212Top1Builder

cpcv, walk_forward, backtesting

Пояснение

TOTAL_SCORE

WF_Return: фактическая доходность WalkForward из последнего сохраненного WF-теста.

WF_Calmar: эффективность доходности относительно просадки в WalkForward.

Robustness_Delta: абсолютное расхождение между медианной оценкой доходности CPCV и доходностью WF.

Sharpe_Stability: прокси-метрика стабильности Sharpe по CPCV; чем ниже, тем лучше.

Backtest_Metric_Wins: количество метрик VectorBT backtest, где модель оказалась лучшей.

TOTAL_SCORE: агрегированный рейтинг стратегии; чем выше, тем лучше.

Система берет последние сохраненные CPCV, WalkForward и Backtesting результаты по каждой модели и строит агрегированный рейтинг.

Используются показатели:

- WF_Return;
- WF_Calmar;
- Robustness_Delta;
- Sharpe_Stability;
- победы по Backtesting-метрикам;
- TOTAL_SCORE.

Модель попадает в сравнение только если для нее есть полный набор сохраненных тестов.

Что сохраняется

Результаты тестов сохраняются в JSON-кэшах:

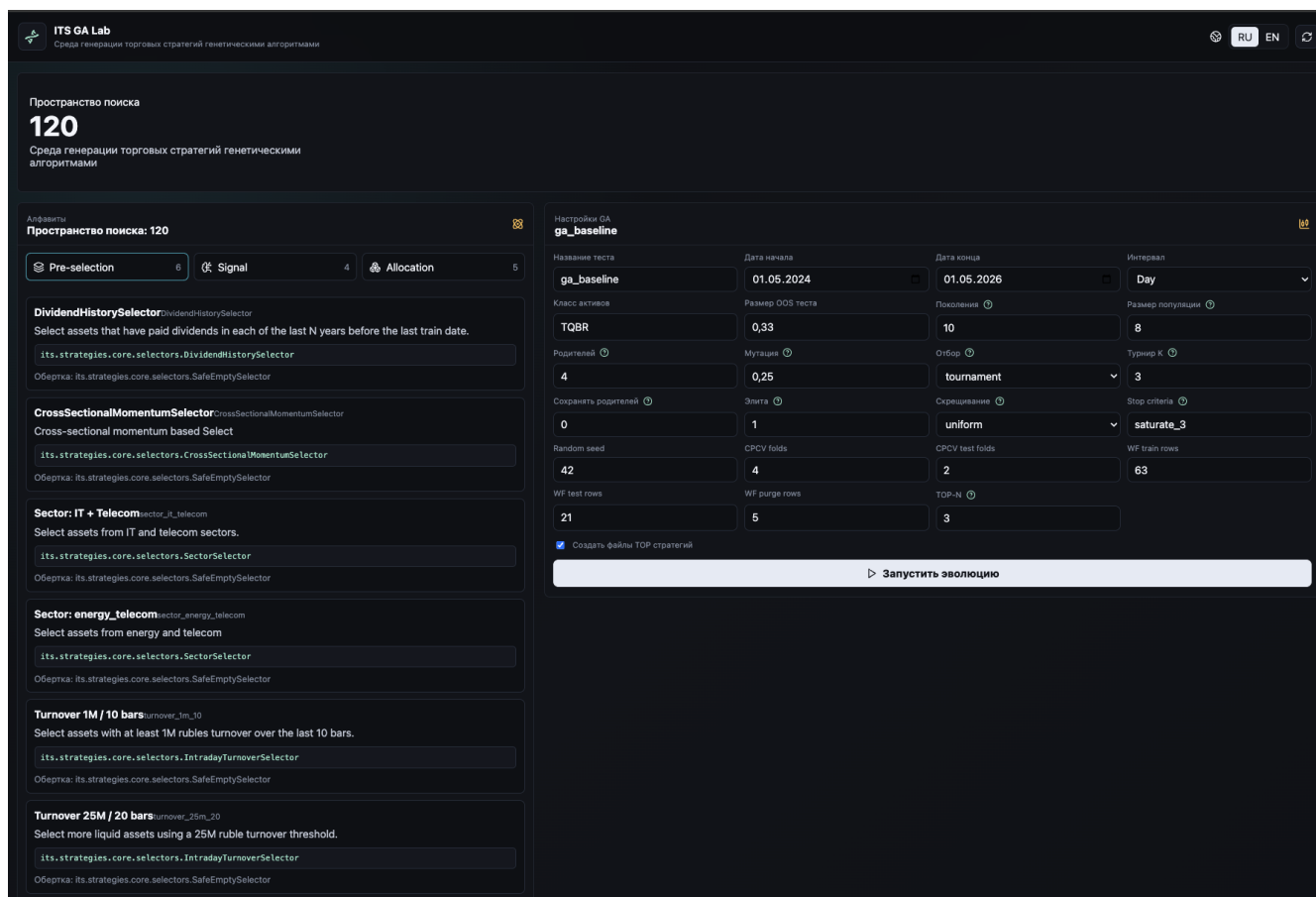
Тип	Переменная окружения	Путь в контейнере
CPCV	STRATEGY_TEST_CACHE_DIR	/app/its/data/strategy_tests/cpcv

Тип	Переменная окружения	Путь в контейнере
WalkForward	STRATEGY_WF_CACHE_DIR	/app/its/data/strategy_tests/walk_for ward
Backtesting	STRATEGY_BACKTEST_CACHE_D IR	/app/its/data/strategy_tests/backtest

Docker Compose сохраняет эти данные в volume `strategy-test-cache`.

GA Lab

GA Lab - подсистема автоматической генерации торговых стратегий с использованием генетических алгоритмов.



Назначение

GA Lab использует уже созданные компоненты системы как алфавит генов:

pre-selection gene + signal gene + allocation gene = candidate strategy

Генетический алгоритм перебирает комбинации, оценивает их на исторических данных и сохраняет лучшие стратегии в кодовую базу.

Основные файлы

Путь	Назначение
its/services/ga_backend	FastAPI backend GA
its/ui/ga-ui	Vue UI GA Lab
its/ga/engine.py	основной GA-движок
its/ga/registry.py	загрузка алфавитов
its/ga/types.py	описание GeneDefinition
its/ga/materialization.py	генерация Python-файлов стратегий

Путь	Назначение
its/ga/alphabets	алфавиты генов
its/strategies/models	папка, куда сохраняются TOP стратегии

Алфавиты

В системе три группы генов.

Pre-selection

Папка:

its/ga/alphabets/pre_selection

Текущие примеры:

- DividendHistorySelector;
- CrossSectionalMomentumSelector;
- sector_it_telecom;
- sector_energy_telecom;
- turnover_1m_10;
- turnover_25m_20.

Signal

Папка:

its/ga/alphabets/signal

Текущие примеры:

- pass_signal;
- PriceBreakoutSignal;
- SMACrossSignal;
- TwoCandlePositiveTrendSignal.

Allocation

Папка:

its/ga/alphabets/allocation

Текущие примеры:

- equal_weighted;
- inverse_volatility;
- HierarchicalRiskParity;
- CVaR;
- CVaRHighRisk.

Пространство поиска

Размер пространства поиска равен произведению количества генов в группах:

$N(\text{pre-selection}) * N(\text{signal}) * N(\text{allocation})$

Каждый кандидат кодируется тремя индексами:

(selector_idx, signal_idx, allocation_idx)

После декодирования кандидат получает имя:

[GA][selector_name][signal_name][allocation_name]

Настройки GA

В UI доступны параметры:

Параметр	Назначение
test_name	имя запуска
start_date, end_date	период данных
interval	интервал свечей
class_code	класс активов, например TQBR
test_size	доля out-of-sample части
num_generations	количество поколений
sol_per_pop	размер популяции
num_parents_mating	число родителей
mutation_probability	вероятность мутации
parent_selection_type	метод отбора
k_tournament	размер турнира
keep_parents	сохранение родителей
keep_elitism	число элитных решений
crossover_type	тип скрещивания
mutation_type	тип мутации
stop_criteria	ранняя остановка PyGAD
random_seed	воспроизводимость
cpcv_n_folds	CPCV folds внутри оценки
cpcv_n_test_folds	CPCV test folds
wf_train_size	train rows для WalkForward
wf_test_size	test rows для WalkForward
wf_purged_size	purge rows
top_n	сколько лучших стратегий вернуть
materialize_top	создавать ли файлы TOP стратегий

Оценка кандидатов

Каждая стратегия-кандидат строится как pipeline:

ga_pre_selection -> ga_signal -> ga_allocation

Далее GA-движок:

- 1 Загружает данные через Data Backend.
- 2 Строит матрицу доходностей.
- 3 Делит период на train и test.
- 4 Обучает pipeline на train.
- 5 Оценивает кандидата на test через CPCV и WalkForward.
- 6 Извлекает метрики.
- 7 Считает итоговый TOTAL_SCORE.

Метрики fitness

В текущей версии используются:

Метрика	Направление	Вес
WF_Return	выше лучше	0.30
WF_Calmar	выше лучше	0.30
Robustness_Delta	ниже лучше	0.20
Sharpe_Stability	ниже лучше	0.20

Также применяются штрафы:

- за максимальную просадку выше порога;
- за слишком узкий портфель, если активных активов мало.

Итоговая шкала:

$TOTAL_SCORE = 100 * weighted_score - penalties$

Результаты запуска

GA Lab отображает:

- статус запуска;
- поколения;
- лучший score;
- средний score;
- лучшего кандидата;
- популяцию;
- TOP стратегий;
- материализованные файлы;
- ошибки материализации, если они возникли.

Материализация стратегий

Если включен `materialize_top`, TOP-N стратегий сохраняются в:

`its/strategies/models`

Для каждого кандидата создается Python-файл вида:

```
ga_generated_<run_id>_top_<rank>.py
```

Также обновляется:

```
its/strategies/models/___init___py
```

После этого стратегия становится видимой в Strategy Lab как обычная зарегистрированная модель.

Кэши запусков

Запуски GA сохраняются в JSON:

```
/app/its/data/ga_runs
```

В Docker Compose это volume:

```
ga-cache
```

Практический сценарий

- 1 Модельер создает новые компоненты: селекторы, сигналы, аллокаторы.
- 2 Добавляет их в GA-алфавиты.
- 3 Открывает GA Lab.
- 4 Настраивает период, размер популяции и число поколений.
- 5 Запускает эволюцию.
- 6 Смотрит TOP стратегий.
- 7 Материализует лучшие стратегии.
- 8 Открывает Strategy Lab.
- 9 Запускает полноценные тесты CPCV, WalkForward и Backtesting.
- 10 Сравнивает новые модели с существующими.

Руководство модельера и разработчика компонентов

Этот раздел описывает, как расширять систему новыми источниками данных, компонентами моделей, готовыми стратегиями и GA-генами.

Общий принцип

Модельер должен описать только предметную логику:

- как отбирать активы;
- как формировать сигнал;
- как распределять веса;
- как управлять позицией.

Инфраструктурные задачи берет на себя платформа:

- загрузка данных;
- подготовка матриц;
- запуск тестов;
- построение отчетов;
- регистрация в UI;
- сравнение результатов;
- генерация новых комбинаций через GA.

Типы компонентов

Selector

Селектор реализует предварительную фильтрацию активов.

Базовый тип:

[its/strategies/core/types/selectors_types.py](#)

Класс должен наследоваться от `Selectros` и реализовать `fit`. Внутри `fit` необходимо выставить:

```
self.to_keep_ = ...
```

`to_keep_` - булева маска активов.

Signal

Сигнальная модель реализует дополнительный отбор после pre-selection.

Базовый тип:

[its/strategies/core/types/signals_types.py](#)

Класс наследуется от `Siglans` и также выставляет `to_keep_`.

Allocation

Аллокатор определяет веса портфеля. В текущей реализации используются skfolio-аллокаторы и собственные расширения:

its/strategies/core/optimization

Аллокатор должен быть совместим с pipeline и выдавать веса через интерфейс skfolio.

Создание нового селектора

- 1 Создать файл в:

its/strategies/core/selectors

- 1 Реализовать класс.

Минимальная структура:

```
import numpy as np
import sklearn.utils.validation as skv

from its.strategies.core.types.selectors_types import Selectros

class MySelector(Selectros):
    """Описание логики селектора."""

    def __init__(self, my_parameter: int = 10):
        self.my_parameter = my_parameter

    def fit(self, X, y=None):
        X = skv.validate_data(self, X, ensure_all_finite="allow-nan")
        self.to_keep_ = np.ones(X.shape[1], dtype=bool)
        return self
```

- 1 Добавить класс в:

its/strategies/core/selectors/__init__.py

- 1 Добавить имя класса в `__all__`.

После этого Strategy Backend сможет отобразить компонент в реестре.

Создание нового сигнала

- 1 Создать файл в:

its/strategies/core/signals

- 1 Наследоваться от `Siglans`.
- 2 Реализовать `fit`.
- 3 Добавить импорт и `__all__` в:

its/strategies/core/signals/__init__.py

Создание нового аллокатора

- 1 Создать или расширить файл в:

its/strategies/core/optimization

- 1 Если используется skfolio, желательно сохранить совместимость с `BaseOptimization`.
- 2 Добавить экспорт в:

its/strategies/core/optimization/__init__.py

Создание модели ядра стратегии

Готовые модели хранятся в:

its/strategies/models

Модель должна наследоваться от `StrategyBuilder`.

Пример структуры:

```
from typing import override

from its.strategies.core.optimization import EqualWeighted
from its.strategies.core.selectors import IntradayTurnoverSelector
from its.strategies.core.types.strategy_types import Pipeline, Strategy, StrategyBuilder

class MyStrategyBuilder(StrategyBuilder):
    """Краткое описание стратегии."""

    @override
    def build(self) -> Strategy:
        return Strategy(
            name="MyStrategy",
            description="Описание модели",
            pipeline=Pipeline(
                steps=[
                    (
                        "pre_selection",
                        IntradayTurnoverSelector(
                            asset_universe_prices=self._asset_universe_prices,
                            lookback_bars=10,
                            min_turnover=1_000_000,
                        ),
                    ),
                    ("allocation", EqualWeighted()),
                ]
            ),
        )
```

Регистрация:

- 1 Импортировать класс в `its/strategies/models/__init__.py`.
- 2 Добавить имя класса в `__all__`.

После этого модель появится во вкладке "Ядро торговой стратегии".

Создание полноценной trading strategy

Полная торговая стратегия находится на уровень выше ядра: она добавляет правила жизненного цикла позиции.

Базовые типы:

`its/strategies_model/core/trading_strategy.py`

Нужно наследоваться от `TradingStrategyBuilder` и реализовать:

- `name`;
- `description`;
- `build_core_strategy`;
- при необходимости `build_exit_policy`;
- при необходимости `build_metadata`.

Пример:

```
from typing import override

from its.strategies.models.top_turnover_eq import ModelTurnoverWithEQBuilder
from its.strategies_model.core.trading_strategy import (
    FixedStopTakeProfitPolicy,
    TradingStrategyBuilder,
)

class MyTradingStrategyBuilder(TradingStrategyBuilder):
    @property
```

```

@override
def name(self) -> str:
    return "MyTradingStrategy"

@property
@override
def description(self) -> str:
    return "Core strategy with fixed stop loss and take profit."

@override
def build_core_strategy(self):
    return ModelTurnoverWithEQBuilder(
        self._asset_universe_prices,
        self._assets_info,
        self._runtime_context,
        self._dividends_info,
    ).build()

@override
def build_exit_policy(self):
    return FixedStopTakeProfitPolicy(
        stop_loss_pct=0.01,
        take_profit_pct=0.03,
    )

```

Регистрация выполняется в:

`its/strategies_model/model/__init__.py`

Добавление компонента в GA

GA использует `GeneDefinition`.

Файл:

`its/ga/types.py`

Минимальный пример гена:

```

from its.ga.types import GeneDefinition
from its.strategies_core.signals import KeepAllSignal

GENES = [
    GeneDefinition(
        id="pass_signal",
        title="Pass-signal",
        description="Do not apply an additional signal filter.",
        group="signal",
        component=KeepAllSignal,
    )
]

```

Группы:

```

pre_selection
signal
allocation

```

Гены размещаются в соответствующих папках:

```

its/ga/alphabets/pre_selection
its/ga/alphabets/signal
its/ga/alphabets/allocation

```

Если компоненту нужны runtime-данные, используется `runtime_args`:

```

GeneDefinition(
    id="DividendHistorySelector",
    title="DividendHistorySelector",
    description="Select assets with dividend history.",
    group="pre_selection",
    component=DividendHistorySelector,
    wrapper=SafeEmptySelector,
    runtime_args={"dividends_df": "dividends_info"},
)

```

Runtime context GA

GA передает компонентам:

Ключ	Содержимое
<code>asset_universe_prices</code>	длинная таблица свечей
<code>assets_info</code>	справочник активов
<code>dividends_info</code>	дивиденды

Рекомендации к компонентам

- Не использовать будущие данные при расчете признаков.
- Проверять входные данные и выдавать понятные ошибки.
- Использовать `SafeEmptySelector`, если компонент может выбрать ноль активов.
- Оставлять docstring: Strategy UI показывает описание из кода.
- Делать параметры явными в `__init__`.
- Не менять глобальное состояние.
- Сохранять совместимость с pandas DataFrame и sklearn/skfolio pipeline.
- Покрывать новую логику тестами.

Тестирование новых компонентов

Тесты находятся в:

`tests`

Рекомендуемые команды:

```
poetry run pytest
poetry run ruff check .
poetry run mypy .
```

В текущем проекте используются тесты для:

- загрузчиков данных;
- селекторов;
- сигналов;
- аллокаторов;
- стратегий;
- GA engine;
- trading strategy backtest.

Методики тестирования и сравнения моделей

ITS содержит три основных типа проверки моделей:

- 1 CPCV.
- 2 WalkForward.
- 3 Backtesting.

Также реализовано агрегированное сравнение стратегий по последним сохраненным тестам.

Общий поток данных для тестов

```

flowchart TD
    Model["Выбранная модель"] --> Data["Data Backend<br/>stocks + prices + dividends"]
    Data --> Matrix["Матрица доходностей или close prices"]
    Matrix --> Fit["Fit pipeline"]
    Fit --> Test["CPCV / WalkForward / Backtesting"]
    Test --> Report["JSON отчет"]
    Report --> UI["Графики и таблицы Strategy UI"]
    Report --> Cache["Кэш тестов"]
  
```

Подготовка данных

Для CPCV и WalkForward используется матрица доходностей:

- 1 Берутся поля `time`, `ticker`, `close`.
- 2 Данные разворачиваются в wide matrix: строки - даты, колонки - тикеры.
- 3 Пропуски заполняются forward fill.
- 4 Удаляются активы с полностью пустыми или постоянными ценами.
- 5 Считается `pct_change`.
- 6 Первый пропуск заполняется нулем.

Для Backtesting используется матрица close prices, а для полноценных trading strategies также high/low prices для stop-loss и take-profit.

CPCV

CPCV - Combinatorial Purged Cross-Validation. Методика нужна для оценки устойчивости стратегии и снижения риска переобучения.

В системе используется:

`skfolio.model_selection.CombinatorialPurgedCV`

Параметры:

Параметр	Смысл
<code>n_folds</code>	число фолдов
<code>n_test_folds</code>	число тестовых фолдов в комбинации
<code>start_date</code> , <code>end_date</code>	период данных
<code>interval</code>	интервал свечей
<code>class_code</code>	класс активов

Что показывает CPCV:

- разброс результатов по путям;
- медианную доходность;
- среднюю доходность;
- стандартное отклонение доходности;
- стабильность Sharpe;
- количество тестовых путей;
- графики cumulative returns по путям.

Когда использовать:

- при первичной проверке новой модели;
- когда нужно понять, насколько стратегия чувствительна к разбиению данных;
- перед запуском более дорогого WalkForward или Backtesting.

WalkForward

WalkForward проверяет стратегию на последовательных временных окнах.

В системе используется:

`skfolio.model_selection.WalkForward`

Параметры:

Параметр	Смысл
<code>test_size</code>	доля OOS-периода после первичного train split
<code>train_size_months</code>	размер train-окна
<code>freq_months</code>	частота сдвига окна
<code>wf_test_size</code>	размер test-участка

Что показывает WalkForward:

- окна train/test;
- метрики по OOS-периодам;
- отдельные кривые окон;
- склеенную OOS equity curve;
- итоговую OOS доходность.

Когда использовать:

- для проверки поведения стратегии во времени;
- для оценки стабильности на последовательных рыночных режимах;
- для отбора стратегий перед Backtesting.

Backtesting

Backtesting моделирует историческую торговлю с ребалансировками, комиссиями, проскальзыванием и начальным капиталом.

В системе используется:

vectorbt

Параметры:

Параметр	Смысл
trading_start_date	дата начала торговли
rebalance_freq	частота ребаланса, например 3ME
rebalance_on	правило выбора даты ребаланса
init_cash	начальный капитал
fees	комиссии
slippage	проскальзывание
tax_rate	налоговая ставка для расчетного after-tax результата
rolling_window	окно rolling Sharpe

Backtesting возвращает:

- итоговую доходность;
- доходность после налога;
- максимальную просадку;
- длительность максимальной просадки;
- equity curve;
- drawdown curve;
- rolling Sharpe;
- таблицу vectorbt stats;
- веса портфеля на каждой ребалансировке;
- состав портфеля;
- секторную агрегацию;
- события исполнения stop-loss и take-profit.

Тестирование full trading strategy

Для моделей из `its/strategies_model/model` Backtesting дополнительно учитывает:

- high/low цены;
- правила выхода из позиции;
- `FixedStopTakeProfitPolicy`;
- события `stop_loss` и `take_profit`;
- цену входа;
- цену исполнения;
- доходность закрытой позиции.

Сравнение стратегий

Сравнение выполняется endpoint:

`GET /api/strategies/comparison/latest`

Система берет последние сохраненные:

- CPCV;
- WalkForward;
- Backtesting.

Модель пропускается, если у нее нет полного набора тестов.

Метрики сравнения

Метрика	Источник	Интерпретация
WF_Return	WalkForward	реализованная OOS доходность
WF_Calmar	WalkForward	доходность относительно просадки
Robustness_Delta	CPCV + WF	расхождение между CPCV median return и WF return
Sharpe_Stability	CPCV	прокси стабильности Sharpe
Backtest_Metric_Wins	Backtesting	число побед по backtest-метрикам
TOTAL_SCORE	comparison	итоговый рейтинг

Метрики Backtesting, где выбираются победители

Для части метрик большее значение лучше:

- Total Return;
- Sharpe Ratio;
- Sortino Ratio;
- Calmar Ratio;
- Omega Ratio;
- Win Rate [%];
- Profit Factor;
- Expectancy.

Для части метрик меньшее значение лучше:

- Max Drawdown;
- Total Fees Paid.

Практическая интерпретация

Стратегия считается более перспективной, если:

- WalkForward показывает положительную OOS-доходность;

- Calmar Ratio выше альтернатив;
- разница между CPCV и WalkForward небольшая;
- разброс результатов по CPCV умеренный;
- Backtesting не показывает чрезмерную просадку;
- портфель не состоит из слишком малого числа активов;
- результаты не зависят от одного удачного периода.

Ограничения

Результаты тестов не являются гарантией будущей доходности. Историческое тестирование отражает только поведение на доступных данных и зависит от качества данных, выбранных параметров, комиссий, ликвидности и рыночных режимов.

API и интеграции

Все внешние запросы проходят через `nginx-gateway`. Ниже указаны публичные пути, доступные после запуска на `localhost:8080`.

Swagger

API	URL
Data Backend	<code>/api/data/docs</code>
Strategy Backend	<code>/api/strategies/docs</code>
GA Backend	<code>/api/ga/docs</code>
Execution Backend	<code>/api/execution/docs</code>
Tech System Backend	<code>/api/tech/docs</code>
Event Log Backend	<code>/api/event-log/docs</code>

Большинство бизнес-endpoints требуют JWT Bearer token, полученный через Tech System (POST `/api/tech/auth/login` или POST `/api/tech/auth/register`). Health-check endpoints доступны без бизнес-параметров.

Data API

Базовый путь:

`/api/data/`

Health

GET `/api/data/health`

Ответ:

`{"status": "ok"}`

Источники

GET `/api/data/sources`

Возвращает список подключенных источников и доступных ресурсов.

Акции

GET `/api/data/stocks`

Основные параметры:

Параметр	Описание
<code>class_code</code>	класс инструментов, по умолчанию <code>TQBR</code>
<code>search</code>	поиск по тикеру или названию
<code>tickers</code>	список тикеров
<code>exchange</code>	фильтр биржи
<code>sector</code>	фильтр сектора
<code>country_of_risk</code>	страна риска

Параметр	Описание
limit, offset	пагинация

Валюты

GET /api/data/currencies

Возвращает валютные инструменты.

Универсальный справочник инструментов

GET /api/data/instruments

Поддерживает поиск и фильтрацию по instrument_types, class_code, exchange, currency, api_trade_available, limit, offset.

Цены

GET /api/data/prices

Основные параметры:

Параметр	Описание
figis	список FIGI
tickers	список тикеров
class_code	класс инструментов
instrument_type	stocks или currencies
start_date	дата начала
end_date	дата окончания
interval	интервал свечей
is_complete	только завершённые свечи

Custom gold bars

GET /api/data/custom-gold-bars

Дополнительные параметры:

Параметр	Описание
gold_ticker	тикер золота, по умолчанию GLDRUB_TOM
gold_class_code	класс золота, по умолчанию CETS
count	количество золотых единиц
bar_type	тип золотого бара

Дивиденды

GET /api/data/dividends

Параметры:

- figis;
- tickers;

- `class_code`;
- `start_date`;
- `end_date`.

Monte Carlo и RSS

```
GET /api/data/monte-carlo
GET /api/data/rss
GET /api/data/rss/sources
POST /api/data/rss/load
```

monte-carlo строит сценарии цены закрытия для одного инструмента. RSS endpoints читают, фильтруют и загружают новости из настроенных источников.

Strategy API

Базовый путь:

```
/api/strategies/
```

Health

```
GET /api/strategies/health
```

Реестр

```
GET /api/strategies/registry
```

Возвращает группы:

- `pre_selection`;
- `signal_model`;
- `allocation`;
- `strategy_model`;
- `trading_strategy_model`.

Модели ядра

```
GET /api/strategies/models
GET /api/strategies/models/{model_name}
```

Детальный endpoint возвращает:

- описание модели;
- состав pipeline;
- component groups;
- доступные отчеты.

Полные торговые стратегии

```
GET /api/strategies/trading-strategies
GET /api/strategies/trading-strategies/{strategy_name}
```

Production state торговых стратегий

```
GET /api/strategies/trading-strategies/prod-ready
PUT /api/strategies/trading-strategies/{strategy_name}/prod-ready
GET /api/strategies/strategy-type
```

Эти endpoints отмечают trading strategy как готовую к Execution, возвращают список таких стратегий и описывают тип полноценной стратегии.

CPCV

```
GET /api/strategies/models/{model_name}/cpcv/tests
GET /api/strategies/models/{model_name}/cpcv/tests/{test_name}
POST /api/strategies/models/{model_name}/cpcv/run
```

Тело **POST**:

```
{
  "test_name": "baseline",
  "start_date": "2023-01-01",
  "end_date": "2025-12-31",
  "interval": "CANDLE_INTERVAL_DAY",
  "class_code": "TQBR",
  "n_folds": 10,
  "n_test_folds": 6
}
```

WalkForward

```
GET /api/strategies/models/{model_name}/walk-forward/tests
GET /api/strategies/models/{model_name}/walk-forward/tests/{test_name}
POST /api/strategies/models/{model_name}/walk-forward/run
```

Тело **POST**:

```
{
  "test_name": "baseline",
  "start_date": "2023-01-01",
  "end_date": "2025-12-31",
  "interval": "CANDLE_INTERVAL_DAY",
  "class_code": "TQBR",
  "test_size": 0.33,
  "train_size_months": 3,
  "freq_months": 3,
  "wf_test_size": 1
}
```

Backtesting

```
GET /api/strategies/models/{model_name}/backtest/tests
GET /api/strategies/models/{model_name}/backtest/tests/{test_name}
POST /api/strategies/models/{model_name}/backtest/run
```

Тело **POST**:

```
{
  "test_name": "baseline",
  "start_date": "2023-01-01",
  "end_date": "2025-12-31",
  "interval": "CANDLE_INTERVAL_DAY",
  "class_code": "TQBR",
  "trading_start_date": "2023-06-01",
  "rebalance_freq": "3ME",
  "rebalance_on": "last",
  "init_cash": 1000000,
  "fees": 0.0008,
  "slippage": 0.0,
  "freq": "1D",
  "rolling_window": 252,
  "tax_rate": 0.13
}
```

Backtesting для full trading strategy

```
GET /api/strategies/trading-strategies/{strategy_name}/backtest/tests
GET /api/strategies/trading-strategies/{strategy_name}/backtest/tests/{test_name}
POST /api/strategies/trading-strategies/{strategy_name}/backtest/run
```

Сравнение

```
GET /api/strategies/comparison/latest
```

GA API

Базовый путь:

/api/ga/

Health

GET /api/ga/health

Алфавиты

GET /api/ga/alphabets

Возвращает группы генов и размер пространства поиска.

Список запусков

GET /api/ga/runs

Детали запуска

GET /api/ga/runs/{run_id}

Запуск GA

POST /api/ga/runs

Пример тела:

```
{
  "test_name": "ga_baseline",
  "start_date": "2023-01-01",
  "end_date": "2025-12-31",
  "interval": "CANDLE_INTERVAL_DAY",
  "class_code": "TQBR",
  "test_size": 0.33,
  "num_generations": 10,
  "sol_per_pop": 8,
  "num_parents_mating": 4,
  "mutation_probability": 0.25,
  "parent_selection_type": "tournament",
  "k_tournament": 3,
  "keep_parents": 0,
  "keep_elitism": 1,
  "crossover_type": "uniform",
  "mutation_type": "random",
  "stop_criteria": "saturate_3",
  "random_seed": 42,
  "cpcv_n_folds": 4,
  "cpcv_n_test_folds": 2,
  "wf_train_size": 63,
  "wf_test_size": 21,
  "wf_purged_size": 5,
  "top_n": 3,
  "materialize_top": true
}
```

Execution API

Базовый путь:

/api/execution/

Health

GET /api/execution/health

Ответ содержит брокера, наличие токена, число настроенных счетов и `order_submission_mode`.

Счета и обзор

GET /api/execution/accounts

GET /api/execution/accounts/{account_id}/overview?operations_days=30

Возвращает настроенные счета T-Invest, портфель, позиции, лимиты вывода, маржинальные атрибуты, активные заявки, stop-заявки и операции.

Заявки

```
POST /api/execution/accounts/{account_id}/orders
POST /api/execution/accounts/{account_id}/stop-orders
```

Обычная заявка содержит `instrument_id`, `side`, `order_type`, `quantity`, `price`, `time_in_force` и комментарий. Stop-заявка содержит `stop_order_type`, `stop_price`, optional `limit_price` и `expire_at`. В режиме `stub` заявка валидируется локально и не отправляется брокеру.

Рыночные данные и стратегии

```
GET /api/execution/market-data/last-price?instrument_id=...
WebSocket /api/execution/ws/orderbook
GET /api/execution/accounts/{account_id}/strategies
PUT /api/execution/accounts/{account_id}/strategies/{strategy_name}
DELETE /api/execution/accounts/{account_id}/strategies/{strategy_name}
POST /api/execution/accounts/{account_id}/strategies/{strategy_name}/runs
```

Назначение стратегии сохраняется в БД. Запуск использует параметры периода, интервала, `order_type`, `limit_offset_pct` и `min_order_value`.

Tech System API

Базовый путь:

```
/api/tech/
```

Основные группы:

```
POST /api/tech/auth/register
POST /api/tech/auth/login
POST /api/tech/auth/refresh
GET /api/tech/auth/me
POST /api/tech/auth/logout
GET /api/tech/profile/me
GET /api/tech/roles
GET /api/tech/permissions
GET /api/tech/users
GET /api/tech/role-requests
GET /api/tech/audit/auth
GET /api/tech/audit/roles
```

Tech System отвечает за регистрацию, JWT access/refresh tokens, роли, permissions, заявки на роли, блокировку пользователей и аудит auth/RBAC-событий.

Event Log API

Базовый путь:

```
/api/event-log/
```

```
GET /api/event-log/health
GET /api/event-log/events
GET /api/event-log/events/filter-options
```

`events` поддерживает фильтры `id`, `date_time_from`, `date_time_to`, `service`, `user`, `http_action`, `ip_address`, `path`, `header`, `body`, `limit`, `offset`.

Интеграционные зависимости

Strategy Backend, GA Backend и Execution Backend используют Data Backend для рыночных данных. Execution Backend также обращается к T-Invest broker API для счетов и заявок. Все backend-сервисы устанавливают Event Log и Observability middleware, а защищенные endpoints используют JWT/RBAC из Tech System.

Execution и технические сервисы

Назначение

Execution и Tech System закрывают контур после исследования стратегии: управление пользователями, доступом, брокерскими счетами, назначением trading strategies и контролируемой отправкой заявок. Эти модули не заменяют промышленный риск-менеджмент, но дают рабочий интерфейс для проверки стратегии на реальном брокерском контуре или в безопасном stub режиме.

Execution

Execution состоит из:

Компонент	Путь	Назначение
Backend	its/services/execution_backend	API счетов, заявок, order book и запусков стратегий
Domain	its/execution	схемы заявок, конфигурация, сервис T-Invest и strategy runner
UI	ui/execution-ui	интерфейс счетов, портфеля, заявок и назначенных стратегий

Основные возможности:

- чтение списка настроенных счетов T-Invest;
- обзор счета: портфель, позиции, заявки, stop-заявки, операции, лимиты и маржинальные атрибуты;
- создание market/limit заявок и stop-loss/take-profit stop-заявок;
- получение последней цены и WebSocket order book;
- назначение prod-ready trading strategies на счет;
- запуск стратегии с расчетом заявок по параметрам периода, интервала, order_type, limit_offset_pct и min_order_value.

Ключевые переменные:

```
EXECUTION_TINVEST_ACCOUNT_IDS=account_id_1,account_id_2
EXECUTION_TINVEST_ACCOUNTS=account_id_1:Main,account_id_2:IIS
EXECUTION_ORDER_SUBMISSION_MODE=stub
```

Для демонстраций используйте stub: заявка валидируется, но не отправляется брокеру. В режиме real backend отправляет заявку через T-Invest API.

Tech System

Tech System отвечает за доступ к защищенным функциям:

- регистрация и вход;
- access/refresh JWT tokens;
- роли, permissions и назначение ролей пользователям;
- заявки на роли и их согласование;
- аудит событий входа и RBAC.

UI доступен по `/tech/auth/`, API - по `/api/tech/`. Execution UI использует эти токены и перенаправляет пользователя на вход, если сессия отсутствует или истекла.

Event Log

Event Log сохраняет пользовательские и API-действия в отдельную БД `event-log-postgres`. API `/api/event-log/events` поддерживает фильтрацию по сервису, пользователю, HTTP-методу, пути, IP, заголовкам, телу запроса и периоду. Tech System UI использует этот API для просмотра журнала.

Observability

Профиль `observability` добавляет Prometheus, Grafana, OpenSearch, Fluent Bit, OpenTelemetry Collector и GlitchTip. Backend-сервисы публикуют `/metrics`, пишут JSON-логи и отправляют ошибки в GlitchTip при включенных переменных окружения.

Эксплуатация, конфигурация и передача системы

Состав поставки

Минимальная поставка системы включает:

- исходный код репозитория;
- `docker-compose.yml`;
- Dockerfile для каждого сервиса;
- frontend-приложения;
- backend-сервисы;
- Python-ядро стратегий;
- GA-движок;
- Execution-модуль;
- Tech System с RBAC/JWT;
- Event Log и observability-конфигурацию;
- Markdown-документацию в `docs`;
- PDF-версии документации в `docs/pdf`;
- скриншоты интерфейсов в `docs/img`;
- тесты в `tests`.

Основная команда запуска

```
docker compose up --build
```

Команда является основной точкой входа для пользователя и заказчика.

PDF-версия документации

Markdown-файлы в `docs` являются основным редактируемым источником документации. Для передачи документации единым файлом система содержит готовые PDF-версии:

- `docs/pdf/its_documentation_ru.pdf`;
- `docs/pdf/its_documentation_en.pdf`.

В web-интерфейсе документации кнопка `PDF` скачивает файл на текущем языке интерфейса. После изменения Markdown-документации PDF можно пересобрать командой:

```
poetry run python scripts/build_docs_pdf.py
```

Скрипт использует Markdown-файлы и изображения из `docs/img`, формирует русскую и английскую версии PDF и сохраняет результат в `docs/pdf`.

Переменные окружения

Переменная	Назначение	Значение по умолчанию
<code>tinvest_token</code>	токен T-Invest	пусто

Переменная	Назначение	Значение по умолчанию
TINVEST_TOKEN	альтернативное имя токена	пусто
TINKOFF_INVEST_API_TOKEN	альтернативное имя токена	пусто
EXECUTION_TINVEST_TOKEN	отдельный токен T-Invest для Execution	пусто
EXECUTION_TINVEST_ACCOUNT_IDS	список счетов Execution через запятую	пусто
EXECUTION_TINVEST_ACCOUNTS	список счетов с именами, например id:Main	пусто
EXECUTION_ORDER_SUBMISSION_MODE	режим заявок: <code>real</code> или <code>stub</code>	<code>real</code>
DATA_BACKEND_STOCKS_TTL_MINUTES	TTL справочника инструментов	<code>30</code>
DATA_BACKEND_BASE_URL	адрес Data Backend для внутренних сервисов	задается в compose
STRATEGY_TEST_CACHE_DIR	кэш CPCV	<code>/app/its/data/strategy_tests/cpcv</code>
STRATEGY_WF_CACHE_DIR	кэш WalkForward	<code>/app/its/data/strategy_tests/walk_forward</code>
STRATEGY_BACKTEST_CACHE_DIR	кэш Backtesting	<code>/app/its/data/strategy_tests/backtest</code>
GA_RUN_CACHE_DIR	кэш GA-запусков	<code>/app/its/data/ga_runs</code>
GA_MODELS_DIR	директория материализации стратегий	<code>/app/its/strategies/models</code>
AUTH_JWT_SECRET_KEY	секрет JWT для Tech System и защищенных API	dev-secret в compose
AUTH_ACCESS_TOKEN_TTL_MINUTES	время жизни access token	<code>30</code>
AUTH_REFRESH_TOKEN_TTL_DAYS	время жизни refresh token	<code>7</code>
EVENT_LOG_DATABASE_URL	БД журнала событий	задается в compose
OBSERVABILITY_ENABLED	общий флаг observability middleware	<code>true</code>
OBSERVABILITY_TRACING_ENABLED	трассировка через OTEL	<code>false</code>
SENTRY_DSN	DSN GlitchTip/Sentry	задается в compose
ITS_GATEWAY_PORT	внешний порт gateway	<code>8080</code>

Хранение данных

В Docker Compose используются volumes:

```
t-invest-cache  
strategy-test-cache  
ga-cache  
postgres-data  
event-log-postgres-data  
prometheus-data  
grafana-data  
opensearch-data
```

Назначение:

- не терять загруженные данные при перезапуске контейнеров;
- не повторять дорогие обращения к источникам;
- сохранять тесты между сессиями;
- сохранять историю GA-запусков;
- хранить пользователей, роли, назначения стратегий и журнал событий;
- хранить состояние observability-инструментов.

Материализованные стратегии

GA Backend монтирует:

```
./its/strategies/models/app/its/strategies/models
```

Это значит, что TOP стратегии, созданные GA, появляются в рабочей копии проекта как обычные Python-файлы.

После генерации рекомендуется:

- 1 Просмотреть созданный файл.
- 2 Проверить импорт в `its/strategies/models/__init__.py`.
- 3 Запустить Strategy Lab и убедиться, что модель появилась в реестре.
- 4 Провести CPCV, WalkForward и Backtesting.
- 5 При необходимости зафиксировать изменения в системе контроля версий.

Логи

Логи можно смотреть стандартными средствами Docker:

```
docker compose logs -f  
docker compose logs -f data-backend  
docker compose logs -f strategy-backend  
docker compose logs -f ga-backend  
docker compose logs -f execution-backend  
docker compose logs -f tech-system-backend  
docker compose logs -f event-log-backend
```

Observability-профиль

Для запуска мониторинга:

```
docker compose --profile observability up --build
```

После запуска доступны:

- Grafana: <http://localhost:8080/grafana/>;
- OpenSearch API: <http://localhost:8080/opensearch-api/>;
- OpenSearch Dashboards: <http://localhost:5601/app/home>;

- GlitchTip: <http://localhost:8001/>.

Backend-сервисы публикуют `/metrics`, пишут структурные JSON-логи и отправляют ошибки в GlitchTip, если включены соответствующие `env`-переменные.

Обновление системы

Рекомендуемый порядок:

- 1 Остановить контейнеры.
- 2 Получить новую версию кода.
- 3 Проверить `.env`.
- 4 Запустить:

```
docker compose up --build
```

- 1 Проверить `/health` и UI.

Резервное копирование

Для сохранения результатов работы нужно учитывать:

- `its/strategies/models` - сгенерированные стратегии;
- Docker volume `strategy-test-cache` - сохраненные тесты;
- Docker volume `ga-cache` - GA-запуски;
- Docker volume `t-invest-cache` - кэш данных;
- Docker volume `postgres-data` - пользователи, роли и назначения стратегий;
- Docker volume `event-log-postgres-data` - журнал событий;
- `observability volumes` - метрики, dashboards и индексы логов;
- `.env` - локальные секреты, не передавать публично.

Безопасность

Текущая конфигурация ориентирована на локальный или закрытый исследовательский контур.

Важно:

- не публиковать T-Invest токен;
- не публиковать `AUTH_JWT_SECRET_KEY`;
- не коммитить `.env`;
- не открывать gateway в публичный интернет без дополнительной аутентификации;
- использовать `EXECUTION_ORDER_SUBMISSION_MODE=stub` для демонстраций и проверок без реальных заявок;
- проверять список счетов `EXECUTION_TINVEST_ACCOUNT_IDS` перед включением `real` режима;
- учитывать, что CORS в backend разрешен широко для удобства разработки;
- ограничить доступ к серверу, если система запускается на удаленной машине.

Ограничения текущей версии

Текущая версия:

- не является полноценным брокерским терминалом с промышленным риск-менеджментом;
- может отправлять заявки через Execution в `real` режиме, но не заменяет independent risk checks;
- поддерживает `stub` режим для безопасной проверки заявок без отправки брокеру;
- не гарантирует будущую доходность;
- зависит от доступности и качества внешнего источника данных.

Перед production-использованием нужно отдельно настроить лимиты риска, права пользователей, регламент подтверждения заявок и monitoring/alerting.

Передача покупателю

При передаче системы рекомендуется предоставить:

- репозиторий или архив кода;
- инструкцию запуска;
- этот комплект документации;
- описание необходимых токенов и прав доступа;
- описание настроенных Execution-счетов и режима заявок;
- перечень известных ограничений;
- примеры тестовых запусков;
- список сгенерированных и базовых стратегий;
- результаты демонстрационных CPCV, WalkForward и Backtesting тестов.

Проверка после передачи

Контрольный сценарий:

- 1 Создать `.env` с токеном.
- 2 Выполнить `docker compose up --build`.
- 3 Открыть Launchpad.
- 4 Перейти в Data Hub и загрузить котировки `SBER`.
- 5 Перейти в Strategy Lab и открыть базовую модель.
- 6 Запустить короткий Backtesting.
- 7 Перейти в GA Lab и открыть список алфавитов.
- 8 Запустить небольшой GA-поиск на малом числе поколений.
- 9 Убедиться, что TOP стратегия создана в `its/strategies/models`.
- 10 Перейти в Tech System, зарегистрироваться или войти.
- 11 Перейти в Execution и проверить список настроенных счетов.
- 12 В `stub` режиме создать тестовую заявку и убедиться, что она не отправлена брокеру.

Научная база и публикации автора

ITS разработана автором как практический результат работы над научной диссертацией. Система реализует переход от теоретической модели торговых стратегий к программной платформе, где стратегии описываются через стандартизированные компоненты, тестируются и могут автоматически генерироваться.

Методологическая основа

Научная база системы включает:

- компонентное представление торговых стратегий;
- архитектурное проектирование с использованием C4-подхода;
- унификацию протоколов торговых моделей;
- разделение ядра стратегии и правил исполнения;
- использование генетических алгоритмов для поиска комбинаций компонентов;
- применение методов проверки устойчивости стратегий;
- создание программной основы для эмпирической апробации.

Связь с диссертационной работой

Глава 3 диссертационного исследования посвящена апробации модели формирования торговых стратегий. В рамках этой главы выполнен переход от теоретического анализа к практической реализации.

Ключевые положения, отраженные в ITS:

- разработаны высокоуровневые архитектурные схемы программной системы;
- реализованы протоколы и модели данных для унификации торговых стратегий;
- создана программная база для реализации, тестирования и доработки стратегий;
- преодолен разрыв между теоретическими положениями и практическим применением;
- сформирована база для эмпирической апробации и верификации торговых стратегий;
- создан фундамент для дальнейшего развития алгоритмической торговли.

Научная логика продукта

Система поддерживает следующий исследовательский цикл:

- 1 Формулирование гипотезы.
- 2 Реализация компонента.
- 3 Сборка стратегии.
- 4 CPCV-проверка.
- 5 WalkForward-проверка.
- 6 Backtesting.
- 7 Сравнение моделей.
- 8 Добавление компонента в GA-алфавит.

- 9 Генерация новых комбинаций.
- 10 Повторная эмпирическая проверка.

Публикации автора

- 1 Алиев Б. Н. Анализ доходности российского рынка акций через курс мировых валют // Инновации и инвестиции. - 2021. - № 6. - С. 77-79. - EDN MNHCHO.
- 2 Алиев Б. Н. Эволюция рынка труда и роль искусственного интеллекта на фондовой бирже // Инновации и инвестиции. - 2023. - № 10. - С. 247-252. - EDN XKAOUN.
- 3 Алиев Б. Н. Генетические алгоритмы как инструмент формирования и эволюции торговых стратегий на рынке ценных бумаг // Финансы и кредит. - 2024. - Т. 30, № 4(844). - С. 851-872. - EDN QZDBSH. - DOI 10.24891/фс.30.4.851.
- 4 Алиев Б. Н. Сравнение точности прогноза для классических и альтернативных ценовых баров в IT-компаниях // Научные исследования экономического факультета. Электронный журнал. - 2025. - Т. 17, № 1(55). - С. 22-38. - DOI 10.38050/2078-3809-2025-17-1-22-38. - EDN KFGQLK.
- 5 Алиев Б. Н. Унификация автоматических трейдинговых систем с использованием принципа сегрегации интерфейсов // Финансовая аналитика: проблемы и решения. - 2024. - Т. 17, № 3(369). - С. 359-366. - DOI 10.24891/fa.17.3.359. - EDN WTIPVH.

Практическая значимость

Практическая значимость ITS заключается в том, что система не ограничивается описанием методологии. Она предоставляет работающую программную среду, где можно:

- создавать новые модели;
- проводить эмпирическую проверку;
- сравнивать результаты;
- генерировать новые стратегии;
- использовать программные артефакты как основу для дальнейшей разработки.

Глоссарий

ITS

Intelligent Trading Strategies - интеллектуальная система формирования торговых стратегий.

Data Hub

Подсистема данных. Отвечает за источники данных, инструменты, котировки, дивиденды, custom bars и API данных.

Strategy Lab

Подсистема работы с моделями. Отвечает за реестр компонентов, готовые стратегии, тестирование и сравнение.

GA Lab

Подсистема генерации стратегий генетическим алгоритмом.

Execution

Подсистема работы с брокерскими счетами T-Invest, заявками, stop-заявками, order book и назначением trading strategies на счет.

Tech System

Техническая подсистема регистрации, входа, JWT-сессий, ролей, permissions и аудита auth/RBAC-событий.

Event Log

Append-only журнал пользовательских и API-действий, сохраняемый в отдельной базе event-log-postgres.

Observability

Профиль мониторинга и диагностики: метрики Prometheus, dashboards Grafana, структурные логи, OpenSearch, tracing и GlitchTip.

Launchpad

Главное окно программы с плитками запуска подсистем.

Pre-selection

Первичный отбор активов до сигнальной модели и аллокации.

Signal

Компонент, который формирует торговый сигнал или дополнительный фильтр после pre-selection.

Allocation

Компонент, который рассчитывает веса активов в портфеле.

Strategy Builder

Python-класс, который собирает ядро стратегии и возвращает объект `Strategy`.

Trading Strategy

Полная торговая стратегия: ядро портфельной модели плюс правила исполнения, например stop-loss и take-profit.

RBAC

Role-Based Access Control - разграничение доступа через роли и permissions.

JWT

JSON Web Token - токен access/refresh сессии пользователя.

CPCV

Combinatorial Purged Cross-Validation - метод проверки стратегии с комбинаторными тестовыми фолдами и purging-подходом.

WalkForward

Методика последовательной проверки на временных окнах, где стратегия обучается на train-части и проверяется на последующей test-части.

Backtesting

Историческое моделирование торговли с учетом ребалансировки, комиссий, проскальзывания и капитала.

Rebalance

Периодическое изменение весов портфеля согласно стратегии.

Equity Curve

Кривая стоимости портфеля или накопленной доходности.

Drawdown

Просадка от предыдущего максимума стоимости портфеля.

Sharpe Ratio

Показатель доходности с учетом риска, рассчитанный как отношение средней доходности к стандартному отклонению.

Calmar Ratio

Отношение доходности к максимальной просадке.

CVaR

Conditional Value at Risk - оценка среднего убытка в хвосте распределения при заданном доверительном уровне.

GA

Genetic Algorithm - генетический алгоритм, который генерирует и отбирает стратегии.

GeneDefinition

Описание одного гена GA: компонента, параметров, обертки и runtime-зависимостей.

Alphabet

Набор генов одной группы: pre-selection, signal или allocation.

Materialization

Создание Python-файла стратегии по результатам GA-запуска.

FIGI

Financial Instrument Global Identifier - глобальный идентификатор финансового инструмента.

TQBR

Класс российских акций на Московской бирже, используемый по умолчанию в системе.

CETS

Класс валютных инструментов, используемый для `GLDRUB_TOM` и custom gold bars.

OHLCV

Данные свечи: open, high, low, close, volume.

Custom gold bar

Альтернативный бар, построенный через стоимость заданного количества золота.

Свидетельство о регистрации программы для ЭВМ

РОССИЙСКАЯ ФЕДЕРАЦИЯ



СВИДЕТЕЛЬСТВО

о государственной регистрации программы для ЭВМ

№ 2026665144

Intelligent Trading Strategies (ITS)

Правообладатель: *Алиев Бейлак Намаз оглы (RU)*

Автор(ы): *Алиев Бейлак Намаз оглы (RU)*



Заявка № 2026664220

Дата поступления 02 мая 2026 г.

Дата государственной регистрации

в Реестре программ для ЭВМ 21 мая 2026 г.

Руководитель Федеральной службы
по интеллектуальной собственности

ДОКУМЕНТ ПОДПИСАН ЭЛЕКТРОННОЙ ПОДПИСЬЮ
Сертификат 00a570e4f7ad48d531b4b8818e75f29506
Владелец **Зубов Юрий Сергеевич**
Действителен с 04.09.2025 по 28.11.2026

Ю.С. Зубов